Tina4 for PHP Developers

This is not a framework

Andre van Zuydam

v3.1.0 - 2026

38 features. Zero dependencies.

Table of Contents

Chapter 1: Getting Started with Tina4 PHP
Chapter 2: Routing
Chapter 3: Request & Response
Chapter 4: Templates
Chapter 5: Database
Chapter 6: ORM
Chapter 7: Authentication
Chapter 8: Middleware
Chapter 15: Frontend with tina4css

Chapter 1: Getting Started with Tina4 PHP

1. What Is Tina4 PHP

Tina4 PHP is a zero-dependency web framework for PHP 8.1+ that gives you routing, an ORM, a template engine, authentication, queues, WebSocket, and 70 other features in a single Composer package under 5,000 lines of code.

It is part of the Tina4 family -- four identical frameworks in Python, PHP, Ruby, and Node.js. Everything you learn here transfers directly to any of the other three languages. The project structure is the same. The template syntax is the same. The CLI commands are the same. The `.env` variables are the same.

Tina4 PHP uses `camelCase` for method names (`fetchOne()`, `softDelete()`, `hasMany()`) following PHP community conventions. Class names are `PascalCase`. Constants are `UPPER_SNAKE_CASE`.

By the end of this chapter, you will have a working Tina4 PHP project with an API endpoint and a rendered HTML page.

2. Prerequisites and Installation

What You Need

Before installing Tina4, make sure you have:

1. **PHP 8.1 or later** -- check with:

```
php -v
```

You should see output like:

```
PHP 8.3.4 (cli) (built: Feb 13 2026 09:27:45) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.3.4, Copyright (c) Zend Technologies
    with Zend OPcache v8.3.4, Copyright (c), by Zend Technologies
```

If you see a version lower than 8.1, upgrade PHP first.

2. **Composer** -- PHP's package manager. Check with:

```
composer --version
```

You should see:

```
Composer version 2.7.2 2024-03-11 17:12:18
```

If Composer is not installed, get it from https://getcomposer.org.

3. **The Tina4 CLI** -- a Rust-based binary that manages all four Tina4 frameworks:

```
# macOS (Homebrew)
brew install tina4stack/tap/tina4

# Linux / macOS (install script)
curl -fsSL https://raw.githubusercontent.com/tina4stack/tina4/main/install.sh | bash

# Windows (PowerShell)
irm https://raw.githubusercontent.com/tina4stack/tina4/main/install.ps1 | iex
```

Verify the CLI is installed:

```
tina4 --version
```

```
tina4 0.1.0
```

4. **Required PHP extensions** -- these ship with most PHP installations:

• `ext-json` (bundled since PHP 8.0)

• `ext-mbstring`

• `ext-openssl`

• `ext-sqlite3` (for the default SQLite database)

• `ext-fileinfo`

Check if they are available:

```
php -m | grep -E "json|mbstring|openssl|sqlite3|fileinfo"
```

```
fileinfo
json
mbstring
openssl
sqlite3
```

If any are missing, install them via your OS package manager (e.g., `apt install php8.3-mbstring` on Ubuntu).

Creating a New Project

Use the Tina4 CLI to scaffold a new project:

```
tina4 init my-store
```

You should see:

```
Creating Tina4 project in ./my-store ...
  Detected language: PHP (composer.json)
  Created .env
  Created .env.example
  Created .gitignore
  Created src/routes/
  Created src/orm/
  Created src/migrations/
  Created src/seeds/
  Created src/templates/
  Created src/templates/errors/
  Created src/public/
  Created src/public/js/
  Created src/public/css/
  Created src/public/scss/
  Created src/public/images/
  Created src/public/icons/
  Created src/locales/
  Created data/
  Created logs/
  Created secrets/
  Created tests/

Project created! Next steps:
  cd my-store
// ...
```

Now install the PHP dependencies:

```
cd my-store
composer install
```

```
Installing dependencies from lock file
  - Installing tina4/tina4-php (v3.0.0): Extracting archive
Generating autoload files
1 package installed
```

That is right -- **one package**. No dependency tree. No version conflicts. Just `tina4/tina4-php`.

Starting the Dev Server

```
tina4 serve
```

```
 _____ _             _  _
|_   _(_)_ __   __ _| || |
  | | | | '_ \ / _` | || |_
  | | | | | | | (_| |__   _|
  |_| |_|_| |_|\__,_|  |_|

  Tina4 PHP v3.0.0
  Server running at http://0.0.0.0:7145
  Debug mode: ON
  Database: sqlite:///data/app.db
  Press Ctrl+C to stop
```

Open your browser to `http://localhost:7145`. You should see the Tina4 welcome page.

Open `http://localhost:7145/health` in your browser or curl it:

```
curl http://localhost:7145/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 12,
  "version": "3.0.0",
  "framework": "tina4-php"
}
```

Your Tina4 PHP project is running.

3. Project Structure Walkthrough

Let us look at what `tina4 init` created:

```
my-store/
■■■ .env                    # Your configuration (gitignored)
■■■ .env.example            # Template for other developers
■■■ .gitignore              # Pre-configured
■■■ composer.json           # Composer package definition
■■■ composer.lock           # Locked dependency versions
■■■ vendor/                 # Composer packages (gitignored)
■■■ src/
■   ■■■ routes/             # Your route handlers go here
■   ■■■ orm/                # Your ORM model classes go here
■   ■■■ migrations/         # SQL migration files
■   ■■■ seeds/              # Database seed files
■   ■■■ templates/          # Frond/Twig templates
■   ■   ■■■ errors/         # Custom 404.html, 500.html
■   ■■■ public/             # Static files (CSS, JS, images)
■   ■   ■■■ js/
■   ■   ■   ■■■ frond.js    # Auto-provided JS helper library
```

```
■   ■   ■■■ css/
■   ■   ■   ■■■ tina4.css   # Built-in CSS utility framework
■   ■   ■■■ scss/
■   ■   ■■■ images/
■   ■   ■■■ icons/
■   ■■■ locales/            # Translation files
■       ■■■ en.json
■■■ data/                   # SQLite databases (gitignored)
// ...
```

Key directories:

• **`src/routes/`** -- Every `.php` file here is auto-loaded at startup. Put your route definitions here. Organize into subdirectories if you want.

• **`src/orm/`** -- Every `.php` file here is auto-loaded. Put your ORM model classes here.

• **`src/templates/`** -- Frond looks here when you call `$response->render("my-page.html", $data)`.

• **`src/public/`** -- Files served directly. `src/public/images/logo.png` is available at `/images/logo.png`.

• **`data/`** -- The default SQLite database (`app.db`) lives here. Gitignored because databases should not be in version control.

4. Your First Route

Let us create an API endpoint that returns a JSON greeting.

Create the file `src/routes/greeting.php`:

```php
<?php
use Tina4\Route;

Route::get("/api/greeting/{name}", function ($request, $response) {
    $name = $request->params["name"];
    return $response->json([
        "message" => "Hello, " . $name . "!",
        "timestamp" => date("c")
    ]);
});
```

Save the file. If the dev server is running with live reload, it picks up the change automatically. If not, restart the server with `tina4 serve`.

Test It

Open your browser to:

```
http://localhost:7145/api/greeting/Alice
```

You should see:

```
{
  "message": "Hello, Alice!",
  "timestamp": "2026-03-22T14:30:00+00:00"
}
```

Or use curl:

```
curl http://localhost:7145/api/greeting/Alice
```

```
{"message":"Hello, Alice!","timestamp":"2026-03-22T14:30:00+00:00"}
```

Notice the difference: the browser shows pretty-printed JSON (because of browser extensions or dev mode), while curl shows compact JSON. You can force pretty output by adding `?pretty=true`:

```
curl "http://localhost:7145/api/greeting/Alice?pretty=true"
```

```
{
  "message": "Hello, Alice!",
  "timestamp": "2026-03-22T14:30:00+00:00"
}
```

Understanding What Happened

1. You created a file in `src/routes/`. Tina4 auto-discovered it at startup.

2. `Route::get("/api/greeting/{name}", ...)` registered a GET route with a path parameter `{name}`.

3. When you requested `/api/greeting/Alice`, the router matched the pattern and called your handler function.

4. `$request->params["name"]` gave you the value `"Alice"` from the URL.

5. `$response->json(...)` serialized the array to JSON, set `Content-Type: application/json`, and returned a `200 OK` response.

Adding More HTTP Methods

Let us add a POST endpoint. Update `src/routes/greeting.php`:

```
<?php
use Tina4\Route;

Route::get("/api/greeting/{name}", function ($request, $response) {
    $name = $request->params["name"];
    return $response->json([
        "message" => "Hello, " . $name . "!",
        "timestamp" => date("c")
    ]);
});

Route::post("/api/greeting", function ($request, $response) {
    $name = $request->body["name"] ?? "World";
    $language = $request->body["language"] ?? "en";

    $greetings = [
        "en" => "Hello",
        "es" => "Hola",
        "fr" => "Bonjour",
        "de" => "Hallo",
        "ja" => "Konnichiwa"
    ];

    $greeting = $greetings[$language] ?? $greetings["en"];

// ...
```

Test the POST endpoint:

```
curl -X POST http://localhost:7145/api/greeting \
  -H "Content-Type: application/json" \
  -d '{"name": "Carlos", "language": "es"}'
```

```
{"message":"Hola, Carlos!","language":"es"}
```

The HTTP status code is `201 Created` (the second argument to `$response->json()`).

5. Your First Template

Tina4 uses the **Frond** template engine -- a zero-dependency, Twig-compatible engine built from scratch. If you have used Twig, Jinja2, or Nunjucks, this will feel familiar.

Create a Base Layout

Create `src/templates/base.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; margin: 0; pad
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
        .product-card { border: 1px solid #ddd; border-radius: 8px; padding: 16px; margin: 8px 0; }
        .product-card h3 { margin-top: 0; }
        .price { color: #2d8f2d; font-weight: bold; font-size: 1.2em; }
        .badge { display: inline-block; padding: 2px 8px; border-radius: 4px; font-size: 0.8em; }
        .badge-success { background: #d4edda; color: #155724; }
        .badge-danger { background: #f8d7da; color: #721c24; }
        nav { background: #333; color: white; padding: 12px 20px; }
        nav a { color: white; text-decoration: none; margin-right: 16px; }
    </style>
</head>
<body>
    <nav>
        <a href="/">Home</a>
        <a href="/products">Products</a>
    </nav>
// ...
```

This base layout defines two blocks (`title` and `content`) that child templates can override. It includes `tina4.css` (the built-in CSS framework) and `frond.js` (the built-in JS helper library).

Create a Product Listing Page

Create `src/templates/products.html`:

```
{% extends "base.html" %}

{% block title %}Products - My Store{% endblock %}

{% block content %}
    <h1>Our Products</h1>
    <p>Showing {{ products | length }} product{{ products | length != 1 ? "s" : "" }}</p>

    {% if products | length > 0 %}
        {% for product in products %}
            <div class="product-card">
                <h3>{{ product.name }}</h3>
                <p>{{ product.description }}</p>
                <p class="price">${{ product.price | number_format(2) }}</p>
                {% if product.in_stock %}
                    <span class="badge badge-success">In Stock</span>
                {% else %}
```

```
                    <span class="badge badge-danger">Out of Stock</span>
                {% endif %}
                {% if not loop.last %}
                    {# Don't add separator after the last item #}
                {% endif %}
            </div>
        {% endfor %}
    {% else %}
// ...
```

Create the Route That Renders the Template

Create `src/routes/pages.php`:

```
<?php
use Tina4\Route;

Route::get("/products", function ($request, $response) {
    $products = [
        [
            "name" => "Wireless Keyboard",
            "description" => "Ergonomic wireless keyboard with backlit keys.",
            "price" => 79.99,
            "in_stock" => true
        ],
        [
            "name" => "USB-C Hub",
            "description" => "7-port USB-C hub with HDMI, SD card reader, and Ethernet.",
            "price" => 49.99,
            "in_stock" => true
        ],
        [
            "name" => "Monitor Stand",
            "description" => "Adjustable aluminum monitor stand with cable management.",
            "price" => 129.99,
            "in_stock" => false
        ],
        [
            "name" => "Mechanical Mouse",
// ...
```

See It in the Browser

Open `http://localhost:7145/products` in your browser. You should see:

• A dark navigation bar at the top with "Home" and "Products" links

• The heading "Our Products"

• A subheading showing "Showing 4 products"

• Four product cards, each with a name, description, price, and stock badge

• The "Monitor Stand" card shows a red "Out of Stock" badge

• The other three show green "In Stock" badges

How Template Rendering Works

1. `$response->render("products.html", ["products" => $products])` tells Frond to render `src/templates/products.html` with the given data.

2. Frond sees `{% extends "base.html" %}` and loads the base template.

3. The `{% block content %}` in `products.html` replaces the same block in `base.html`.

4. `{{ product.name }}` outputs the value, auto-escaped for HTML safety.

5. `{{ product.price | number_format(2) }}` formats the number with 2 decimal places.

6. `{% for product in products %}` loops through the array.

7. `{% if product.in_stock %}` conditionally renders the stock badge.

8. `{{ products | length }}` returns the count of items in the array.

About tina4css

The `tina4.css` file included in the base template is Tina4's built-in CSS utility framework. It provides layout utilities, typography, and common UI patterns without needing Bootstrap or Tailwind. It is auto-provided when you scaffold a project -- you do not need to download it separately.

6. Understanding .env

Open the `.env` file at the root of your project:

```
TINA4_DEBUG=true
```

That is likely all you see. The scaffold creates a minimal `.env` with debug mode enabled. Everything else uses defaults.

The important defaults for development:

| Variable | Default Value | What It Means |

|----------|---------------|---------------|

| `TINA4_PORT` | `7145` | Server runs on port 7145 |

| `DATABASE_URL` | `sqlite:///data/app.db` | SQLite database in the `data/` directory |

| `TINA4_LOG_LEVEL` | `ALL` | All log messages are output |

| `CORS_ORIGINS` | `*` | All origins allowed (fine for development) |

| `TINA4_RATE_LIMIT` | `60` | 60 requests per minute per IP |

To change the port, add it to `.env`:

```
TINA4_DEBUG=true
TINA4_PORT=8080
```

Restart the server (`Ctrl+C`, then `tina4 serve`). It now runs on port 8080.

For the complete `.env` reference with all 68 variables, see Book 0, Chapter 4.

7. The Dev Dashboard

With `TINA4_DEBUG=true`, Tina4 provides a built-in development dashboard. First, enable it by adding a console token to your `.env`:

```
TINA4_DEBUG=true
TINA4_CONSOLE=true
```

```
TINA4_CONSOLE_TOKEN=my-dev-token
```

Restart the server and navigate to:

```
http://localhost:7145/tina4/console
```

Enter your token (`my-dev-token`) when prompted. You will see:

• **System Overview** -- framework version, PHP version, uptime, memory usage, database status

• **Request Inspector** -- recent HTTP requests with method, path, status, duration, and request ID. Click any request to see full headers, body, database queries, and template renders.

• **Error Log** -- unhandled exceptions with stack traces and occurrence counts

• **Queue Manager** -- queue status (pending, reserved, failed, dead-letter messages)

• **WebSocket Monitor** -- active WebSocket connections with metadata

• **Routes** -- all registered routes with their methods, paths, and middleware

The console is a powerful debugging tool. It shows you exactly what your application is doing without adding print statements or log calls to your code.

Additionally, when you visit any HTML page (like `/products`), you will see a **debug overlay** -- a toolbar at the bottom of the page showing:

• Request details (method, URL, duration)

• Database queries executed (with timing)

• Template renders (with timing)

• Session data

• Recent log entries

This overlay is only visible when `TINA4_DEBUG=true`. It is never shown in production.

8. Exercise: Greeting API + Product List Template

Build the following two features from scratch, without looking at the examples above.

Exercise Part A: Greeting API

Create an API endpoint at `GET /api/greet` that:

1. Accepts a query parameter `name` (e.g., `/api/greet?name=Sarah`)

2. If `name` is missing, defaults to `"Stranger"`

3. Returns JSON like:

```
{
  "greeting": "Welcome, Sarah!",
  "time_of_day": "afternoon"
}
```

4. The `time_of_day` should be calculated from the server's current hour:

• 5:00 - 11:59 = "morning"

• 12:00 - 16:59 = "afternoon"

• 17:00 - 20:59 = "evening"

• 21:00 - 4:59 = "night"

Test your endpoint with:

```
curl "http://localhost:7145/api/greet?name=Sarah"
curl "http://localhost:7145/api/greet"
```

Exercise Part B: Product List Page

Create a page at `GET /store` that:

1. Displays a list of at least 5 products (hardcoded for now)

2. Each product has: name, category, price, and a boolean `featured` flag

3. Featured products should be visually highlighted (different background color, border, or badge)

4. The page should show the total number of products and the number of featured products

5. Use template inheritance -- create a layout template and a page template that extends it

6. Include `tina4.css` and `frond.js`

Your products data should look like this in your route handler:

```
$products = [
    ["name" => "Espresso Machine", "category" => "Kitchen", "price" => 299.99, "featured" => true],
    ["name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "featured" => false],
    ["name" => "Standing Desk", "category" => "Office", "price" => 549.99, "featured" => true],
    ["name" => "Noise-Canceling Headphones", "category" => "Electronics", "price" => 199.99, "featured" => tru
    ["name" => "Water Bottle", "category" => "Fitness", "price" => 24.99, "featured" => false]
];
```

Expected browser output:

• A page titled "Our Store"

• Text showing "5 products, 3 featured"

• A list of product cards with name, category, price, and a "Featured" badge on the highlighted items

• Featured products have a distinct visual style (your choice -- different border color, background, star icon, etc.)

9. Solutions

Solution A: Greeting API

Create `src/routes/greet.php`:

```
<?php
use Tina4\Route;

Route::get("/api/greet", function ($request, $response) {
    $name = $request->query["name"] ?? "Stranger";
    $hour = (int) date("G");

    if ($hour >= 5 && $hour < 12) {
        $timeOfDay = "morning";
```

```
    } elseif ($hour >= 12 && $hour < 17) {
        $timeOfDay = "afternoon";
    } elseif ($hour >= 17 && $hour < 21) {
        $timeOfDay = "evening";
    } else {
        $timeOfDay = "night";
    }

    return $response->json([
        "greeting" => "Welcome, " . $name . "!",
        "time_of_day" => $timeOfDay
    ]);
});
```

Test:

```
curl "http://localhost:7145/api/greet?name=Sarah"

{"greeting":"Welcome, Sarah!","time_of_day":"afternoon"}

curl "http://localhost:7145/api/greet"

{"greeting":"Welcome, Stranger!","time_of_day":"afternoon"}
```

Solution B: Product List Page

Create `src/templates/store-layout.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; margin: 0; pad
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
        header { background: #1a1a2e; color: white; padding: 16px 20px; }
        header h1 { margin: 0; }
        .stats { color: #888; margin: 8px 0 20px; }
        .product-grid { display: grid; gap: 16px; }
        .product-card { background: white; border: 2px solid #e0e0e0; border-radius: 8px; padding: 16px; }
        .product-card.featured { border-color: #ffc107; background: #fffdf0; }
        .product-name { font-size: 1.2em; font-weight: bold; margin: 0 0 4px; }
        .product-category { color: #666; font-size: 0.9em; }
        .product-price { color: #2d8f2d; font-weight: bold; font-size: 1.1em; margin-top: 8px; }
        .featured-badge { background: #ffc107; color: #333; padding: 2px 8px; border-radius: 4px; font-size: 
    </style>
</head>
<body>
    <header>
        <h1>{% block header %}Store{% endblock %}</h1>
// ...
```

Create `src/templates/store.html`:

```
{% extends "store-layout.html" %}

{% block title %}Our Store{% endblock %}
{% block header %}Our Store{% endblock %}

{% block content %}
    <p class="stats">{{ products | length }} products, {{ featured_count }} featured</p>

    <div class="product-grid">
```

```
        {% for product in products %}
            <div class="product-card{{ product.featured ? ' featured' : '' }}">
                <p class="product-name">
                    {{ product.name }}
                    {% if product.featured %}
                        <span class="featured-badge">Featured</span>
                    {% endif %}
                </p>
                <p class="product-category">{{ product.category }}</p>
                <p class="product-price">${{ product.price | number_format(2) }}</p>
            </div>
        {% endfor %}
    </div>
{% endblock %}
```

Create `src/routes/store.php`:

```
<?php
use Tina4\Route;

Route::get("/store", function ($request, $response) {
    $products = [
        ["name" => "Espresso Machine", "category" => "Kitchen", "price" => 299.99, "featured" => true],
        ["name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "featured" => false],
        ["name" => "Standing Desk", "category" => "Office", "price" => 549.99, "featured" => true],
        ["name" => "Noise-Canceling Headphones", "category" => "Electronics", "price" => 199.99, "featured" =>
        ["name" => "Water Bottle", "category" => "Fitness", "price" => 24.99, "featured" => false]
    ];

    $featuredCount = count(array_filter($products, fn($p) => $p["featured"]));

    return $response->render("store.html", [
        "products" => $products,
        "featured_count" => $featuredCount
    ]);
});
```

Open `http://localhost:7145/store` in your browser. You should see:

• A dark header reading "Our Store"

• Text showing "5 products, 3 featured"

• Five product cards in a grid

• Three cards (Espresso Machine, Standing Desk, Noise-Canceling Headphones) have a yellow border, light yellow background, and a "Featured" badge

• Two cards (Yoga Mat, Water Bottle) have a standard white background with gray border

• Each card shows the product name, category, and price formatted with two decimal places

10. Gotchas

1. File not auto-discovered

Problem: You created a route file but nothing happens when you visit the URL.

Cause: The file is not in `src/routes/`. Double-check the path. It must be inside `src/routes/` (or a subdirectory of it), and the file must end with `.php`.

Fix: Move the file to `src/routes/your-file.php` and restart the server.

2. "Class not found" errors

Problem: `Class 'Tina4\Route' not found` or similar.

Cause: Missing `use` statement or Composer autoload is out of date.

Fix: Make sure your route file starts with `<?php` and includes `use Tina4\Route;`. Run `composer dump-autoload` if the error persists.

3. JSON response shows HTML

Problem: Your JSON endpoint returns HTML instead of JSON.

Cause: You returned a string instead of using `$response->json()`. If you return a plain string, Tina4 treats it as HTML.

Fix: Always use `$response->json($data)` for JSON endpoints, not `echo json_encode($data)`.

4. Template not found

Problem: `Template "my-page.html" not found` error.

Cause: The template file is not in `src/templates/`, or there is a typo in the filename.

Fix: Check that the file exists at `src/templates/my-page.html`. The name in `$response->render()` is relative to `src/templates/`.

5. Port already in use

Problem: `Error: Address already in use (port 7145)`

Cause: Another process (or another Tina4 instance) is using port 7145.

Fix: Either stop the other process, or change the port:

```
TINA4_PORT=8080
```

Or use the CLI flag: `tina4 serve --port 8080`.

6. Changes not reflected

Problem: You edited a file but the browser shows the old version.

Cause: In some setups, live reload may not be active. Also, browser caching can serve stale versions.

Fix: Hard-refresh the browser (`Ctrl+Shift+R` or `Cmd+Shift+R`). If that does not help, restart the dev server with `Ctrl+C` and `tina4 serve`.

7. .env not loaded

Problem: Environment variables seem to have no effect.

Cause: The `.env` file must be at the project root (same directory as `composer.json`). If it is in a subdirectory, Tina4 will not find it.

Fix: Move `.env` to the project root.

8. Debug mode in production

Problem: Your production site shows stack traces and query details.

Cause: `TINA4_DEBUG=true` in production.

Fix: Set `TINA4_DEBUG=false` in your production `.env`. This hides all debug information, enables HTML minification, and activates `.broken` file health checks.

Chapter 2: Routing

1. How Routing Works in Tina4

Every web application maps URLs to code. You type `/products` in your browser, the framework finds the function that handles `/products`, runs it, and sends back the result. That mapping is called routing.

In Tina4, you define routes in PHP files inside `src/routes/`. Every `.php` file in that directory (and its subdirectories) is auto-loaded when the server starts. You do not need to register files or update a central config -- just drop a file in and it works.

Here is the simplest possible route:

```
<?php
use Tina4\Route;

Route::get("/hello", function ($request, $response) {
    return $response->json(["message" => "Hello, World!"]);
});
```

Save that as `src/routes/hello.php`, start the server with `tina4 serve`, and visit `http://localhost:7145/hello`:

```
{"message":"Hello, World!"}
```

That is it. One line registers the route, one line handles the request.

2. HTTP Methods

Tina4 supports all five standard HTTP methods. Each one has a static method on the `Route` class:

```
<?php
use Tina4\Route;

Route::get("/products", function ($request, $response) {
    return $response->json(["action" => "list all products"]);
});

Route::post("/products", function ($request, $response) {
    return $response->json(["action" => "create a product"], 201);
});

Route::put("/products/{id}", function ($request, $response) {
    $id = $request->params["id"];
    return $response->json(["action" => "replace product " . $id]);
});

Route::patch("/products/{id}", function ($request, $response) {
    $id = $request->params["id"];
    return $response->json(["action" => "update product " . $id]);
});

Route::delete("/products/{id}", function ($request, $response) {
    $id = $request->params["id"];
    return $response->json(["action" => "delete product " . $id]);
});
```

Test each one:

```
curl http://localhost:7145/products
```

```
{"action":"list all products"}
```

```
curl -X POST http://localhost:7145/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Widget"}'
```

```
{"action":"create a product"}
```

```
curl -X PUT http://localhost:7145/products/42
```

```
{"action":"replace product 42"}
```

```
curl -X PATCH http://localhost:7145/products/42
```

```
{"action":"update product 42"}
```

```
curl -X DELETE http://localhost:7145/products/42
```

```
{"action":"delete product 42"}
```

Use `GET` for reading, `POST` for creating, `PUT` for full replacement, `PATCH` for partial updates, and `DELETE` for removal. This follows the REST convention and makes your API predictable.

3. Path Parameters

Path parameters let you capture values from the URL. Wrap the parameter name in curly braces:

```
<?php
use Tina4\Route;

Route::get("/users/{id}/posts/{postId}", function ($request, $response) {
    $userId = $request->params["id"];
    $postId = $request->params["postId"];

    return $response->json([
        "user_id" => $userId,
        "post_id" => $postId
    ]);
});
```

```
curl http://localhost:7145/users/5/posts/99
```

```
{"user_id":"5","post_id":"99"}
```

Notice that `user_id` came back as the string `"5"`, not the integer `5`. Path parameters are always strings by default.

Typed Parameters

You can enforce a type by adding a colon and the type after the parameter name:

```
<?php
use Tina4\Route;

Route::get("/orders/{id:int}", function ($request, $response) {
    $id = $request->params["id"]; // This is now an integer
    return $response->json([
        "order_id" => $id,
        "type" => gettype($id)
    ]);
});
```

```
curl http://localhost:7145/orders/42
```

```
{"order_id":42,"type":"integer"}
```

If you pass a non-integer value, the route will not match and you will get a 404:

```
curl http://localhost:7145/orders/abc
```

```
{"error":"Not found","path":"/orders/abc","status":404}
```

Supported types:

| Type | Matches | Example |

|------|---------|---------|

| `int` | Digits only | `{id:int}` matches `42` but not `abc` |

| `float` | Decimal numbers | `{price:float}` matches `19.99` |

| `alpha` | Letters only | `{slug:alpha}` matches `hello` but not `hello123` |

| `alphanumeric` | Letters and digits | `{code:alphanumeric}` matches `abc123` |

4. Query Parameters

Query parameters are the key-value pairs after the `?` in a URL. Access them via `$request->query`:

```
<?php
use Tina4\Route;

Route::get("/search", function ($request, $response) {
    $q = $request->query["q"] ?? "";
    $page = (int) ($request->query["page"] ?? 1);
    $limit = (int) ($request->query["limit"] ?? 10);

    return $response->json([
        "query" => $q,
        "page" => $page,
        "limit" => $limit,
        "offset" => ($page - 1) * $limit
    ]);
});
```

```
curl "http://localhost:7145/search?q=keyboard&page=2&limit=20"
```

```
{"query":"keyboard","page":2,"limit":20,"offset":20}
```

If a query parameter is missing, `$request->query["key"]` will not exist, so always use the null coalescing operator (`??`) to provide defaults.

5. Route Groups

When you have a set of routes that share a common prefix, use `Route::group()` to avoid repeating yourself:

```
<?php
use Tina4\Route;

Route::group("/api/v1", function () {

    Route::get("/users", function ($request, $response) {
```

```
        return $response->json(["users" => []]);
    });

    Route::get("/users/{id:int}", function ($request, $response) {
        $id = $request->params["id"];
        return $response->json(["user" => ["id" => $id, "name" => "Alice"]]);
    });

    Route::post("/users", function ($request, $response) {
        return $response->json(["created" => true], 201);
    });

    Route::get("/products", function ($request, $response) {
        return $response->json(["products" => []]);
    });
});
```

The routes above register as `/api/v1/users`, `/api/v1/users/{id}`, and `/api/v1/products`. You write short paths inside the group, and Tina4 prepends the prefix automatically.

```
curl http://localhost:7145/api/v1/users

{"users":[]}

curl http://localhost:7145/api/v1/products

{"products":[]}
```

Groups can be nested:

```
<?php
use Tina4\Route;

Route::group("/api", function () {
    Route::group("/v1", function () {
        Route::get("/status", function ($request, $response) {
            return $response->json(["version" => "1.0"]);
        });
    });

    Route::group("/v2", function () {
        Route::get("/status", function ($request, $response) {
            return $response->json(["version" => "2.0"]);
        });
    });
});

curl http://localhost:7145/api/v1/status

{"version":"1.0"}

curl http://localhost:7145/api/v2/status

{"version":"2.0"}
```

6. Middleware

Middleware is code that runs before (or after) your route handler. Use it for authentication, logging, rate limiting, input validation, or anything that should happen on multiple routes.

Middleware on a Single Route

Pass middleware as the third argument to any route method:

```php
<?php
use Tina4\Route;

function logRequest($request, $response, $next) {
    $start = microtime(true);
    error_log("[" . date("Y-m-d H:i:s") . "] " . $request->method . " " . $request->path);

    $result = $next($request, $response);

    $duration = round((microtime(true) - $start) * 1000, 2);
    error_log("  Completed in " . $duration . "ms");

    return $result;
}

Route::get("/api/data", function ($request, $response) {
    return $response->json(["data" => [1, 2, 3]]);
}, "logRequest");
```

The middleware function receives `$request`, `$response`, and `$next`. Call `$next($request, $response)` to continue to the route handler. If you do not call `$next`, the route handler never runs -- useful for blocking unauthorized requests.

Blocking Middleware

Here is middleware that checks for an API key:

```php
<?php
use Tina4\Route;

function requireApiKey($request, $response, $next) {
    $apiKey = $request->headers["X-API-Key"] ?? "";

    if ($apiKey !== "my-secret-key") {
        return $response->json(["error" => "Invalid API key"], 401);
    }

    return $next($request, $response);
}

Route::get("/api/secret", function ($request, $response) {
    return $response->json(["secret" => "The answer is 42"]);
}, "requireApiKey");
```

```
curl http://localhost:7145/api/secret
```

```
{"error":"Invalid API key"}
```

The response status is `401 Unauthorized`.

```
curl http://localhost:7145/api/secret -H "X-API-Key: my-secret-key"
```

```
{"secret":"The answer is 42"}
```

Middleware on a Group

Apply middleware to an entire group by passing it as the third argument to `Route::group()`:

```php
<?php
use Tina4\Route;

function requireAuth($request, $response, $next) {
    $token = $request->headers["Authorization"] ?? "";

    if (empty($token)) {
```

```
        return $response->json(["error" => "Authentication required"], 401);
    }

    return $next($request, $response);
}

Route::group("/api/admin", function () {

    Route::get("/dashboard", function ($request, $response) {
        return $response->json(["page" => "admin dashboard"]);
    });

    Route::get("/users", function ($request, $response) {
        return $response->json(["page" => "user management"]);
    });

}, "requireAuth");
```

Every route inside the group now requires the `Authorization` header. You do not need to add the middleware to each route individually.

Multiple Middleware

Chain multiple middleware by passing an array:

```
Route::get("/api/important", function ($request, $response) {
    return $response->json(["data" => "important stuff"]);
}, ["logRequest", "requireApiKey", "requireAuth"]);
```

Middleware runs in order: `logRequest` first, then `requireApiKey`, then `requireAuth`, then the route handler. If any middleware does not call `$next`, the chain stops there.

7. Route Decorators: @noauth and @secured

Tina4 provides two special decorators for controlling authentication on routes.

@noauth -- Public Routes

When your application has global authentication middleware, use the `@noauth` annotation to mark specific routes as public:

```
<?php
use Tina4\Route;

/**
 * @noauth
 */
Route::get("/api/public/info", function ($request, $response) {
    return $response->json([
        "app" => "My Store",
        "version" => "1.0.0"
    ]);
});
```

The `@noauth` decorator tells Tina4 to skip authentication checks for this route, even if global auth middleware is configured in `.env` or applied to the parent group.

@secured -- Protected GET Routes

The `@secured` annotation explicitly marks a GET route as requiring authentication:

```php
<?php
use Tina4\Route;

/**
 * @secured
 */
Route::get("/api/profile", function ($request, $response) {
    // $request->user is populated by the auth middleware
    return $response->json([
        "user" => $request->user
    ]);
});
```

By default, `POST`, `PUT`, `PATCH`, and `DELETE` routes are considered secured. `GET` routes are not -- they are public unless you add `@secured`. This matches the common pattern where reading data is public but modifying data requires authentication.

8. Wildcard and Catch-All Routes

Wildcard Routes

Use `*` at the end of a path to match anything after it:

```php
<?php
use Tina4\Route;

Route::get("/docs/*", function ($request, $response) {
    $path = $request->params["*"] ?? "";
    return $response->json([
        "section" => "docs",
        "path" => $path
    ]);
});
```

```
curl http://localhost:7145/docs/getting-started
```

```
{"section":"docs","path":"getting-started"}
```

```
curl http://localhost:7145/docs/api/authentication/jwt
```

```
{"section":"docs","path":"api/authentication/jwt"}
```

Catch-All Route (Custom 404)

Register a catch-all to handle any unmatched URL:

```php
<?php
use Tina4\Route;

Route::get("/*", function ($request, $response) {
    return $response->json([
        "error" => "Page not found",
        "path" => $request->path
    ], 404);
});
```

This route should be defined last (or in a file that sorts alphabetically after your other route files) so it does not shadow your real routes. Tina4 matches routes in the order they are registered -- the first match wins.

Alternatively, you can create a custom 404 page by placing a template at `src/templates/errors/404.html`:

```
{% extends "base.html" %}
```

```
{% block title %}Not Found{% endblock %}

{% block content %}
    <h1>404 - Page Not Found</h1>
    <p>The page you are looking for does not exist.</p>
    <a href="/">Go back home</a>
{% endblock %}
```

Tina4 automatically uses this template for any unmatched route when the template file exists.

9. Route Listing via CLI

As your application grows, you will want to see all registered routes at a glance. Use the Tina4 CLI:

```
tina4 routes

Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /hello                        -                   public
GET      /products                     -                   public
POST     /products                     -                   secured
PUT      /products/{id}                -                   secured
PATCH    /products/{id}                -                   secured
DELETE   /products/{id}                -                   secured
GET      /api/v1/users                 -                   public
GET      /api/v1/users/{id:int}        -                   public
POST     /api/v1/users                 -                   secured
GET      /api/admin/dashboard          requireAuth         public
GET      /api/admin/users              requireAuth         public
GET      /api/public/info              -                   @noauth
GET      /api/profile                  -                   @secured
GET      /search                       -                   public
GET      /docs/*                       -                   public
```

The `Auth` column shows whether a route is public, secured (default for non-GET methods), explicitly `@noauth`, or explicitly `@secured`.

You can also filter by method:

```
tina4 routes --method POST

Method   Path                          Middleware          Auth
------   ----                          ----------          ----
POST     /products                     -                   secured
POST     /api/v1/users                 -                   secured
```

Or search for a specific path pattern:

```
tina4 routes --filter users

Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /api/v1/users                 -                   public
GET      /api/v1/users/{id:int}        -                   public
POST     /api/v1/users                 -                   secured
GET      /api/admin/users              requireAuth         public
```

10. Organizing Route Files

You are free to organize route files any way you like. Tina4 loads every `.php` file in `src/routes/` recursively. Here are two common patterns:

Pattern 1: One File Per Resource

```
src/routes/
■■■ products.php     # All product routes
■■■ users.php        # All user routes
■■■ orders.php       # All order routes
■■■ pages.php        # HTML page routes
```

Pattern 2: Subdirectories by Feature

```
src/routes/
■■■ api/
■   ■■■ products.php
■   ■■■ users.php
■   ■■■ orders.php
■■■ admin/
■   ■■■ dashboard.php
■   ■■■ settings.php
■■■ pages/
    ■■■ home.php
    ■■■ about.php
```

Both patterns work identically. The directory structure has no effect on the URL paths -- only the route definitions inside the files matter. Choose whichever pattern keeps your project navigable.

11. Exercise: Build a Full CRUD API for Products

Build a complete REST API for managing products. All data is stored in a PHP array (no database yet -- we will add that in Chapter 5).

Requirements

Create a file `src/routes/product-api.php` with the following routes:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/products` | List all products. Support `?category=` filter. |

| `GET` | `/api/products/{id:int}` | Get a single product by ID. Return 404 if not found. |

| `POST` | `/api/products` | Create a new product. Return 201. |

| `PUT` | `/api/products/{id:int}` | Replace a product. Return 404 if not found. |

| `DELETE` | `/api/products/{id:int}` | Delete a product. Return 204 with no body. |

Each product has: `id` (int), `name` (string), `category` (string), `price` (float), `in_stock` (bool).

Start with this seed data:

```
$products = [
    ["id" => 1, "name" => "Wireless Keyboard", "category" => "Electronics", "price" => 79.99, "in_stock" => tr
    ["id" => 2, "name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "in_stock" => true],
    ["id" => 3, "name" => "Coffee Grinder", "category" => "Kitchen", "price" => 49.99, "in_stock" => false],
    ["id" => 4, "name" => "Standing Desk", "category" => "Office", "price" => 549.99, "in_stock" => true],
    ["id" => 5, "name" => "Running Shoes", "category" => "Fitness", "price" => 119.99, "in_stock" => true]
```

```
];
```

Test with:

```
# List all
curl http://localhost:7145/api/products

# Filter by category
curl "http://localhost:7145/api/products?category=Fitness"

# Get one
curl http://localhost:7145/api/products/3

# Create
curl -X POST http://localhost:7145/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Desk Lamp", "category": "Office", "price": 39.99, "in_stock": true}'

# Update
curl -X PUT http://localhost:7145/api/products/3 \
  -H "Content-Type: application/json" \
  -d '{"name": "Burr Coffee Grinder", "category": "Kitchen", "price": 59.99, "in_stock": true}'

# Delete
curl -X DELETE http://localhost:7145/api/products/3

# Not found
curl http://localhost:7145/api/products/999
```

12. Solution

Create `src/routes/product-api.php`:

```
<?php
use Tina4\Route;

// In-memory product store (resets on server restart)
$products = [
    ["id" => 1, "name" => "Wireless Keyboard", "category" => "Electronics", "price" => 79.99, "in_stock" => tr
    ["id" => 2, "name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "in_stock" => true],
    ["id" => 3, "name" => "Coffee Grinder", "category" => "Kitchen", "price" => 49.99, "in_stock" => false],
    ["id" => 4, "name" => "Standing Desk", "category" => "Office", "price" => 549.99, "in_stock" => true],
    ["id" => 5, "name" => "Running Shoes", "category" => "Fitness", "price" => 119.99, "in_stock" => true]
];

$nextId = 6;

// List all products, optionally filter by category
Route::get("/api/products", function ($request, $response) use (&$products) {
    $category = $request->query["category"] ?? null;

    if ($category !== null) {
        $filtered = array_values(array_filter(
            $products,
            fn($p) => strtolower($p["category"]) === strtolower($category)
        ));
        return $response->json(["products" => $filtered, "count" => count($filtered)]);
    }
// ...
```

Expected output for the test commands:

List all:

```
{"products":[{"id":1,"name":"Wireless Keyboard","category":"Electronics","price":79.99,"in_stock":true},{"id"
```

Filter by category:

```
{"products":[{"id":2,"name":"Yoga Mat","category":"Fitness","price":29.99,"in_stock":true},{"id":5,"name":"Run
```

Get one:

```
{"id":3,"name":"Coffee Grinder","category":"Kitchen","price":49.99,"in_stock":false}
```

Create:

```
{"id":6,"name":"Desk Lamp","category":"Office","price":39.99,"in_stock":true}
```

(Status: `201 Created`)

Update:

```
{"id":3,"name":"Burr Coffee Grinder","category":"Kitchen","price":59.99,"in_stock":true}
```

Delete: empty response with status `204 No Content`.

Not found:

```
{"error":"Product not found","id":999}
```

(Status: `404 Not Found`)

13. Gotchas

1. Trailing Slashes Matter

Problem: `/products` works but `/products/` returns a 404 (or vice versa).

Cause: Tina4 treats `/products` and `/products/` as different routes by default.

Fix: Pick one convention and stick with it. If you want both to work, register the route without a trailing slash -- Tina4 will redirect `/products/` to `/products` automatically when `TINA4_TRAILING_SLASH_REDIRECT=true` is set in `.env`.

2. Parameter Names Must Be Unique in a Path

Problem: `/users/{id}/posts/{id}` does not work as expected -- both parameters have the same name.

Cause: The second `{id}` overwrites the first in `$request->params`.

Fix: Use distinct names: `/users/{userId}/posts/{postId}`.

3. Method Conflicts

Problem: You defined `Route::get("/items/{id}", ...)` and `Route::get("/items/{action}", ...)` and the wrong handler runs.

Cause: Both patterns match `/items/42`. The first one registered wins.

Fix: Use typed parameters to disambiguate: `Route::get("/items/{id:int}", ...)` will only match integers, leaving `/items/export` free for the other route. Alternatively, restructure your paths: `/items/{id:int}` and `/items/actions/{action}`.

4. Route Handler Must Return a Response

Problem: Your route handler runs but the browser shows an empty page or a 500 error.

Cause: You forgot the `return` statement. Without `return`, the handler returns `null` and Tina4 does not know what to send back.

Fix: Always `return $response->json(...)` or `return $response->html(...)` or `return $response->render(...)`. Every handler must return something.

5. Route Files Must Start with `<?php`

Problem: Your route file is ignored. No errors, just no routes registered.

Cause: The file does not start with `<?php`. Without the PHP opening tag, the file is not parsed as PHP code.

Fix: Make sure every route file starts with `<?php` on the very first line.

6. Middleware Function Must Be a String Name

Problem: Passing a closure directly as middleware causes an error.

Cause: Tina4 expects middleware to be referenced by function name (a string), not as an inline closure.

Fix: Define your middleware as a named function and pass the name as a string: `"myMiddleware"`, not `function ($request, $response, $next) { ... }`.

7. Group Prefix Must Start with a Slash

Problem: `Route::group("api/v1", ...)` produces routes like `/api/v1/users` but they do not match.

Cause: The group prefix should start with `/` for consistency. While Tina4 may auto-correct this, it is better to be explicit.

Fix: Always start group prefixes with `/`: `Route::group("/api/v1", ...)`.

Chapter 3: Request & Response

1. The Two Objects You Always Get

Every route handler in Tina4 receives two arguments: `$request` and `$response`. The request tells you what the client sent. The response is how you send something back. Together they are the entire HTTP conversation.

```
<?php
use Tina4\Route;

Route::get("/echo", function ($request, $response) {
    return $response->json([
        "method" => $request->method,
        "path" => $request->path,
        "your_ip" => $request->ip
    ]);
});
```

```
curl http://localhost:7145/echo
```

```
{"method":"GET","path":"/echo","your_ip":"127.0.0.1"}
```

That is the pattern for every route: inspect the request, build the response, return it.

2. The Request Object

The `$request` object gives you access to everything the client sent. Here is the complete list of properties.

method

The HTTP method as an uppercase string: `"GET"`, `"POST"`, `"PUT"`, `"PATCH"`, or `"DELETE"`.

```
$request->method // "GET"
```

path

The URL path without query parameters:

```
// Request to /api/users?page=2
$request->path // "/api/users"
```

params

Path parameters from the URL pattern (see Chapter 2):

```
// Route: /users/{id}/posts/{postId}
// Request: /users/5/posts/42
$request->params["id"]     // "5" (or 5 if typed as {id:int})
$request->params["postId"] // "42"
```

query

Query string parameters as an associative array:

```
// Request: /search?q=keyboard&page=2&sort=price
$request->query["q"]    // "keyboard"
$request->query["page"] // "2"
$request->query["sort"] // "price"
```

body

The parsed request body. For JSON requests, this is an associative array. For form submissions, it contains the form fields:

```
// POST with {"name": "Widget", "price": 9.99}
$request->body["name"]  // "Widget"
$request->body["price"] // 9.99
```

headers

Request headers as an associative array. Header names are normalized to their original casing:

```
$request->headers["Content-Type"]  // "application/json"
$request->headers["Authorization"] // "Bearer eyJhbGci..."
$request->headers["X-Custom"]      // "my-value"
```

ip

The client's IP address:

```
$request->ip // "127.0.0.1"
```

Tina4 respects `X-Forwarded-For` and `X-Real-IP` headers when behind a reverse proxy.

cookies

Cookies sent by the client:

```
$request->cookies["session_id"]  // "abc123"
$request->cookies["preferences"] // "dark-mode"
```

files

Uploaded files (covered in detail in section 7):

```
$request->files["avatar"] // File object with name, type, size, tmpPath
```

Inspecting the Full Request

Here is a route that dumps everything:

```
<?php
use Tina4\Route;

Route::post("/debug/request", function ($request, $response) {
    return $response->json([
        "method" => $request->method,
        "path" => $request->path,
        "params" => $request->params,
        "query" => $request->query,
        "body" => $request->body,
        "headers" => $request->headers,
        "ip" => $request->ip,
        "cookies" => $request->cookies
    ]);
});
```

```
curl -X POST "http://localhost:7145/debug/request?page=1" \
  -H "Content-Type: application/json" \
  -H "X-Custom: hello" \
  -d '{"name": "test"}'
```

```
{
  "method": "POST",
  "path": "/debug/request",
```

```
  "params": {},
  "query": {"page": "1"},
  "body": {"name": "test"},
  "headers": {
    "Content-Type": "application/json",
    "X-Custom": "hello",
    "Host": "localhost:7145",
    "User-Agent": "curl/8.4.0",
    "Accept": "*/*",
    "Content-Length": "16"
  },
  "ip": "127.0.0.1",
  "cookies": {}
}
```

3. The Response Object

The `$response` object is your toolkit for sending data back to the client. Every method on it returns the response, so you can chain calls together.

json() -- JSON Response

The most common response for APIs. Pass any array or value and it becomes JSON:

```
return $response->json(["name" => "Alice", "age" => 30]);
```

```
{"name":"Alice","age":30}
```

Pass a status code as the second argument:

```
return $response->json(["id" => 7, "name" => "Widget"], 201);
```

This returns `201 Created` with the JSON body.

html() -- Raw HTML Response

Return an HTML string directly:

```
return $response->html("<h1>Hello</h1><p>This is HTML.</p>");
```

Sets `Content-Type: text/html; charset=utf-8` automatically.

text() -- Plain Text Response

Return plain text:

```
return $response->text("Just a plain string.");
```

Sets `Content-Type: text/plain; charset=utf-8`.

render() -- Template Response

Render a Frond template with data (covered in depth in Chapter 4):

```
return $response->render("products.html", [
    "products" => $products,
    "title" => "Our Products"
]);
```

Tina4 looks for the template in `src/templates/`, renders it with the provided data, and returns the HTML.

redirect() -- Redirect Response

Send the client to a different URL:

```
return $response->redirect("/login");
```

This sends a `302 Found` redirect by default. Pass a different status code for permanent redirects:

```
return $response->redirect("/new-location", 301);
```

file() -- File Download Response

Send a file to the client for download:

```
return $response->file("/path/to/report.pdf");
```

This sets the appropriate `Content-Type` based on the file extension and adds a `Content-Disposition` header so the browser downloads the file rather than displaying it.

You can set a custom filename:

```
return $response->file("/path/to/report.pdf", "monthly-report-march-2026.pdf");
```

4. Status Codes

Every response method accepts a status code. Here are the most common ones you will use:

| Code | Meaning | When to Use |

|------|---------|-------------|

| `200` | OK | Default. Successful GET, PUT, PATCH. |

| `201` | Created | Successful POST that created a resource. |

| `204` | No Content | Successful DELETE. No body needed. |

| `301` | Moved Permanently | URL has permanently changed. |

| `302` | Found | Temporary redirect. |

| `400` | Bad Request | Invalid input from the client. |

| `401` | Unauthorized | Missing or invalid authentication. |

| `403` | Forbidden | Authenticated but not allowed. |

| `404` | Not Found | Resource does not exist. |

| `409` | Conflict | Duplicate or conflicting data. |

| `422` | Unprocessable Entity | Valid JSON but fails business rules. |

| `500` | Internal Server Error | Something went wrong on the server. |

You can also set the status explicitly with the `status()` method and chain it:

```
return $response->status(201)->json(["id" => 7, "created" => true]);
```

This is equivalent to `$response->json(["id" => 7, "created" => true], 201)` but some developers find the chained form more readable.

5. Custom Headers

Set response headers with the `header()` method:

```
Route::get("/api/data", function ($request, $response) {
    return $response
        ->header("X-Request-Id", uniqid())
        ->header("X-Rate-Limit-Remaining", "57")
        ->header("Cache-Control", "no-cache")
        ->json(["data" => [1, 2, 3]]);
});
```

```
curl -v http://localhost:7145/api/data 2>&1 | grep "< X-"
```

```
< X-Request-Id: 65f3a7b8c1234
< X-Rate-Limit-Remaining: 57
```

Headers are case-insensitive in HTTP, but it is conventional to use `Title-Case` for custom headers and prefix them with `X-`.

CORS Headers

Tina4 handles CORS automatically based on the `CORS_ORIGINS` setting in `.env`. The default `*` allows all origins. For production, restrict it:

```
CORS_ORIGINS=https://myapp.com,https://admin.myapp.com
```

You rarely need to set CORS headers manually, but you can if needed:

```
return $response
    ->header("Access-Control-Allow-Origin", "https://myapp.com")
    ->header("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE")
    ->header("Access-Control-Allow-Headers", "Content-Type, Authorization")
    ->json(["data" => "value"]);
```

6. Cookies

Set cookies on the response:

```
Route::post("/login", function ($request, $response) {
    // After validating credentials...
    return $response
        ->cookie("session_id", "abc123xyz", [
            "httpOnly" => true,
            "secure" => true,
            "sameSite" => "Strict",
            "maxAge" => 3600,       // 1 hour in seconds
            "path" => "/"
        ])
        ->json(["message" => "Logged in"]);
});
```

Cookie options:

| Option | Type | Default | Description |

|--------|------|---------|-------------|

| `httpOnly` | bool | `false` | Cannot be accessed by JavaScript |

| `secure` | bool | `false` | Only sent over HTTPS |

| `sameSite` | string | `"Lax"` | `"Strict"`, `"Lax"`, or `"None"` |

| `maxAge` | int | session | Lifetime in seconds |

| `path` | string | `"/"` | URL path scope |

| `domain` | string | current | Domain scope |

Read cookies from the request:

```
Route::get("/profile", function ($request, $response) {
    $sessionId = $request->cookies["session_id"] ?? null;

    if ($sessionId === null) {
        return $response->json(["error" => "Not logged in"], 401);
    }

    return $response->json(["session" => $sessionId]);
});
```

Delete a cookie by setting its `maxAge` to `0`:

```
return $response
    ->cookie("session_id", "", ["maxAge" => 0, "path" => "/"])
    ->json(["message" => "Logged out"]);
```

7. File Uploads

Uploaded files are available via `$request->files`. Each file is an object with properties for the file's metadata and a temporary path.

Handling a Single File Upload

```
<?php
use Tina4\Route;

Route::post("/api/upload", function ($request, $response) {
    if (empty($request->files["image"])) {
        return $response->json(["error" => "No file uploaded"], 400);
    }

    $file = $request->files["image"];

    return $response->json([
        "name" => $file->name,        // "photo.jpg"
        "type" => $file->type,        // "image/jpeg"
        "size" => $file->size,        // 245760 (bytes)
        "tmp_path" => $file->tmpPath  // Temporary file location
    ]);
});
```

Test with curl:

```
curl -X POST http://localhost:7145/api/upload \
  -F "image=@/path/to/photo.jpg"

{
  "name": "photo.jpg",
  "type": "image/jpeg",
  "size": 245760,
  "tmp_path": "/tmp/tina4_upload_abc123"
}
```

Saving the Uploaded File

The uploaded file is stored in a temporary location. Move it to a permanent location:

```
<?php
use Tina4\Route;

Route::post("/api/upload", function ($request, $response) {
    if (empty($request->files["image"])) {
        return $response->json(["error" => "No file uploaded"], 400);
    }

    $file = $request->files["image"];

    // Validate file type
    $allowedTypes = ["image/jpeg", "image/png", "image/gif", "image/webp"];
    if (!in_array($file->type, $allowedTypes)) {
        return $response->json(["error" => "Invalid file type. Allowed: JPEG, PNG, GIF, WebP"], 400);
    }

    // Validate file size (max 5MB)
    $maxSize = 5 * 1024 * 1024;
    if ($file->size > $maxSize) {
        return $response->json(["error" => "File too large. Maximum size: 5MB"], 400);
    }

    // Generate a unique filename
    $extension = pathinfo($file->name, PATHINFO_EXTENSION);
    $filename = uniqid("img_") . "." . $extension;
// ...
```

```
curl -X POST http://localhost:7145/api/upload \
  -F "image=@/path/to/photo.jpg"
```

```
{
  "message": "File uploaded successfully",
  "filename": "img_65f3a7b8c1234.jpg",
  "url": "/uploads/img_65f3a7b8c1234.jpg",
  "size": 245760
}
```

The uploaded file is now available at `http://localhost:7145/uploads/img_65f3a7b8c1234.jpg`.

Handling Multiple Files

When the HTML form uses `multiple` or you have multiple file inputs:

```
Route::post("/api/upload-many", function ($request, $response) {
    $results = [];

    foreach ($request->files as $key => $file) {
        $extension = pathinfo($file->name, PATHINFO_EXTENSION);
        $filename = uniqid("file_") . "." . $extension;
        $destination = __DIR__ . "/../../public/uploads/" . $filename;

        if (!is_dir(dirname($destination))) {
            mkdir(dirname($destination), 0755, true);
        }

        rename($file->tmpPath, $destination);

        $results[] = [
            "original_name" => $file->name,
            "saved_as" => $filename,
            "url" => "/uploads/" . $filename
        ];
```

```
    }

    return $response->json(["uploaded" => $results, "count" => count($results)], 201);
});
```

8. File Downloads

Send files to the client using `$response->file()`:

```
<?php
use Tina4\Route;

Route::get("/api/reports/{filename}", function ($request, $response) {
    $filename = $request->params["filename"];
    $filepath = __DIR__ . "/../../data/reports/" . $filename;

    if (!file_exists($filepath)) {
        return $response->json(["error" => "Report not found"], 404);
    }

    return $response->file($filepath);
});
```

The browser will download the file. Tina4 automatically detects the MIME type from the file extension and sets the appropriate headers.

To force a specific download filename:

```
return $response->file($filepath, "Q1-2026-Sales-Report.pdf");
```

9. Content Negotiation

Sometimes the same endpoint should return different formats based on what the client asks for. Check the `Accept` header:

```
<?php
use Tina4\Route;

Route::get("/api/products/{id:int}", function ($request, $response) {
    $id = $request->params["id"];
    $product = [
        "id" => $id,
        "name" => "Wireless Keyboard",
        "price" => 79.99
    ];

    $accept = $request->headers["Accept"] ?? "application/json";

    if (strpos($accept, "text/html") !== false) {
        return $response->render("product-detail.html", ["product" => $product]);
    }

    if (strpos($accept, "text/plain") !== false) {
        $text = "Product #" . $id . ": " . $product["name"] . " - $" . $product["price"];
        return $response->text($text);
    }

    // Default to JSON
    return $response->json($product);
```

```
});

# JSON (default)
curl http://localhost:7145/api/products/1

{"id":1,"name":"Wireless Keyboard","price":79.99}

# Plain text
curl http://localhost:7145/api/products/1 -H "Accept: text/plain"

Product #1: Wireless Keyboard - $79.99

# HTML (would render the template)
curl http://localhost:7145/api/products/1 -H "Accept: text/html"

<!DOCTYPE html>
<html>...rendered template...</html>
```

10. Exercise: Build an Image Upload API

Build an API that handles image uploads and serves them back. Create two endpoints:

Requirements

| Method | Path | Description |

|--------|------|-------------|

| `POST` | `/api/images` | Upload an image. Validate type and size. Return the image URL. |

| `GET` | `/api/images/{filename}` | Return the uploaded image file. Return 404 if not found. |

Rules:

1. Only accept JPEG, PNG, and WebP files

2. Maximum file size: 2MB

3. Save files to `src/public/uploads/` with a unique filename

4. Return the original filename, the saved filename, file size in KB, and the URL

5. The GET endpoint should serve the file directly (not JSON)

Test with:

```
# Upload
curl -X POST http://localhost:7145/api/images \
  -F "image=@/path/to/photo.jpg"

# Download
curl http://localhost:7145/api/images/img_65f3a7b8c1234.jpg --output downloaded.jpg
```

11. Solution

Create `src/routes/images.php`:

```
<?php
use Tina4\Route;

Route::post("/api/images", function ($request, $response) {
```

```
    // Check if a file was uploaded
    if (empty($request->files["image"])) {
        return $response->json(["error" => "No image file provided. Use field name 'image'."], 400);
    }

    $file = $request->files["image"];

    // Validate file type
    $allowedTypes = ["image/jpeg", "image/png", "image/webp"];
    if (!in_array($file->type, $allowedTypes)) {
        return $response->json([
            "error" => "Invalid file type",
            "received" => $file->type,
            "allowed" => $allowedTypes
        ], 400);
    }

    // Validate file size (max 2MB)
    $maxSize = 2 * 1024 * 1024;
    if ($file->size > $maxSize) {
        return $response->json([
// ...
```

Expected output for upload:

```
{
  "message": "Image uploaded successfully",
  "original_name": "photo.jpg",
  "saved_name": "img_65f3a7b8c1234.jpg",
  "size_kb": 240.0,
  "type": "image/jpeg",
  "url": "/uploads/img_65f3a7b8c1234.jpg"
}
```

(Status: `201 Created`)

Expected output for invalid type:

```
{
  "error": "Invalid file type",
  "received": "application/pdf",
  "allowed": ["image/jpeg", "image/png", "image/webp"]
}
```

(Status: `400 Bad Request`)

Expected output for file too large:

```
{
  "error": "File too large",
  "size_bytes": 5242880,
  "max_bytes": 2097152
}
```

(Status: `400 Bad Request`)

The GET endpoint returns the raw image file with the correct `Content-Type` header. The browser displays the image directly, and curl with `--output` saves it to disk.

12. Gotchas

1. Forgetting `return`

Problem: Your handler runs (you can see log output) but the browser shows an empty response or a 500 error.

Cause: You wrote `$response->json([...])` without `return`.

Fix: Always write `return $response->json([...])`. The response object needs to be returned from the handler for Tina4 to send it to the client.

2. Body Is Null for JSON Requests

Problem: `$request->body` is `null` or empty even though you are sending JSON.

Cause: You forgot the `Content-Type: application/json` header in your request. Without it, Tina4 does not know to parse the body as JSON.

Fix: Always include `-H "Content-Type: application/json"` when sending JSON with curl. In frontend JavaScript, `fetch()` with `JSON.stringify()` requires `headers: {"Content-Type": "application/json"}`.

3. Content-Type Mismatch

Problem: You call `$response->json()` but the client receives HTML, or you call `$response->html()` but the client gets plain text.

Cause: A middleware or error handler is overwriting the response. Or you are returning a string instead of using a response method.

Fix: Make sure your handler returns `$response->json(...)`, `$response->html(...)`, or another response method. Do not use `echo` -- it bypasses the response object entirely.

4. File Uploads Return Empty

Problem: `$request->files` is empty even though you are uploading a file.

Cause: The form is not using `enctype="multipart/form-data"`, or the curl command is using `-d` instead of `-F`.

Fix: For HTML forms, use `<form enctype="multipart/form-data">`. For curl, use `-F "field=@file.jpg"` (with `@`), not `-d`.

5. Redirect Loops

Problem: The browser shows "too many redirects" or hangs.

Cause: You have a route that redirects to another route, which redirects back to the first one. For example, `/login` redirects to `/dashboard`, and `/dashboard` redirects to `/login` because the user is not authenticated.

Fix: Check your redirect logic carefully. Use the browser's network inspector to trace the redirect chain. Make sure your auth check does not redirect authenticated users away from pages they should access.

6. Cookie Not Set

Problem: You called `$response->cookie(...)` but the browser does not show the cookie.

Cause: If `secure` is `true`, the cookie is only sent over HTTPS. During local development with `http://localhost`, the cookie is silently dropped.

Fix: Set `"secure" => false` during development, or use `"secure" => ($_ENV["TINA4_DEBUG"] ?? "false") !== "true"` to auto-switch based on environment.

7. Large Request Body Rejected

Problem: POST requests with large bodies return a 413 error.

Cause: The request body exceeds the configured maximum size.

Fix: Increase `TINA4_MAX_BODY_SIZE` in `.env`. The default is `10mb`. For file upload endpoints, you may need `50mb` or more:

```
TINA4_MAX_BODY_SIZE=50mb
```

Chapter 4: Templates

1. Why Templates

In Chapter 1, you saw `$response->render("products.html", $data)` and it produced a full HTML page. That rendering was done by **Frond**, Tina4's built-in template engine. Frond is a zero-dependency, Twig-compatible engine that ships with the framework. If you have used Twig, Jinja2, or Nunjucks, you already know 90% of Frond.

Templates live in `src/templates/`. When you call `$response->render("page.html", $data)`, Frond loads `src/templates/page.html`, processes the tags and expressions, and returns the final HTML.

This chapter covers every feature of the template engine so you can build real pages.

2. Variables and Expressions

Output a variable with double curly braces:

```
<h1>Hello, {{ name }}!</h1>
```

Route handler:

```
<?php
use Tina4\Route;

Route::get("/welcome", function ($request, $response) {
    return $response->render("welcome.html", [
        "name" => "Alice"
    ]);
});
```

Create `src/templates/welcome.html`:

```
<!DOCTYPE html>
<html>
<head><title>Welcome</title></head>
<body>
    <h1>Hello, {{ name }}!</h1>
</body>
</html>
```

Expected browser output:

```
Hello, Alice!
```

Accessing Nested Data

Use dot notation for nested arrays:

```
$data = [
    "user" => [
        "name" => "Alice",
        "email" => "alice@example.com",
        "address" => [
            "city" => "Cape Town",
            "country" => "South Africa"
        ]
    ]
];
```

```
return $response->render("profile.html", $data);
```

```
<p>{{ user.name }} lives in {{ user.address.city }}, {{ user.address.country }}.</p>
```

Output:

```
Alice lives in Cape Town, South Africa.
```

Expressions

You can use basic expressions inside `{{ }}`:

```
<p>Total: ${{ price * quantity }}</p>
<p>Discounted: ${{ price * 0.9 }}</p>
<p>Full name: {{ first_name ~ " " ~ last_name }}</p>
```

The `~` operator concatenates strings.

3. Template Inheritance

Template inheritance is the most powerful feature. Define a base layout once, then extend it in every page.

Base Layout

Create `src/templates/base.twig`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My App{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    {% block head %}{% endblock %}
</head>
<body>
    <nav>
        <a href="/">Home</a>
        <a href="/about">About</a>
        <a href="/contact">Contact</a>
    </nav>

    <main>
        {% block content %}{% endblock %}
    </main>

    <footer>
        <p>&copy; 2026 My App. All rights reserved.</p>
    </footer>

    <script src="/js/frond.js"></script>
// ...
```

This base defines four blocks: `title`, `head`, `content`, and `scripts`. Child templates override only the blocks they need.

Child Template

Create `src/templates/about.twig`:

```
{% extends "base.twig" %}
```

```
{% block title %}About Us{% endblock %}

{% block content %}
    <h1>About Us</h1>
    <p>We have been building things since {{ founded_year }}.</p>
    <p>Our team has {{ team_size }} members across {{ office_count }} offices.</p>
{% endblock %}
```

Route handler:

```
<?php
use Tina4\Route;

Route::get("/about", function ($request, $response) {
    return $response->render("about.twig", [
        "founded_year" => 2020,
        "team_size" => 12,
        "office_count" => 3
    ]);
});
```

Expected browser output: A full HTML page with the nav, the "About Us" content, and the footer. The `<title>` tag says "About Us". The `head` and `scripts` blocks are empty because the child template did not override them.

Using `{{ parent() }}`

Sometimes you want to add to a block rather than replace it entirely. Use `{{ parent() }}`:

```
{% extends "base.twig" %}

{% block head %}
    {{ parent() }}
    <link rel="stylesheet" href="/css/contact-form.css">
{% endblock %}

{% block content %}
    <h1>Contact Us</h1>
    <form>...</form>
{% endblock %}
```

The `head` block now contains everything from the base template's `head` block plus the extra stylesheet.

4. Includes

Break your templates into reusable pieces with `{% include %}`:

Create `src/templates/partials/header.twig`:

```
<header>
    <div class="logo">{{ site_name | default("My App") }}</div>
    <nav>
        <a href="/">Home</a>
        <a href="/products">Products</a>
        <a href="/contact">Contact</a>
    </nav>
</header>
```

Create `src/templates/partials/product-card.twig`:

```
<div class="product-card{{ product.featured ? ' featured' : '' }}">
    <h3>{{ product.name }}</h3>
```

```
    <p class="price">${{ product.price | number_format(2) }}</p>
    {% if product.in_stock %}
        <span class="badge-success">In Stock</span>
    {% else %}
        <span class="badge-danger">Out of Stock</span>
    {% endif %}
</div>
```

Use them in a page:

```
{% extends "base.twig" %}

{% block content %}
    {% include "partials/header.twig" %}

    <h1>Products</h1>
    {% for product in products %}
        {% include "partials/product-card.twig" %}
    {% endfor %}
{% endblock %}
```

The `product` variable is automatically available inside the included template because it exists in the current scope (the for loop).

Passing Variables to Includes

You can pass specific variables with `with`:

```
{% include "partials/header.twig" with {"site_name": "Cool Store"} %}
```

Use `only` to isolate the included template from the parent scope:

```
{% include "partials/header.twig" with {"site_name": "Cool Store"} only %}
```

With `only`, the included template can only access `site_name` -- it cannot see any other variables from the parent.

5. For Loops

Loop through arrays with `{% for %}`:

```
<ul>
{% for item in items %}
    <li>{{ item }}</li>
{% endfor %}
</ul>
```

The `loop` Variable

Inside a for loop, Frond provides a special `loop` variable:

| Property | Type | Description |

|----------|------|-------------|

| `loop.index` | int | Current iteration (1-based) |

| `loop.index0` | int | Current iteration (0-based) |

| `loop.first` | bool | True on the first iteration |

| `loop.last` | bool | True on the last iteration |

| `loop.length` | int | Total number of items |

| `loop.revindex` | int | Iterations remaining (1-based) |

```
<table>
    <thead>
        <tr><th>#</th><th>Name</th><th>Price</th></tr>
    </thead>
    <tbody>
    {% for product in products %}
        <tr class="{{ loop.index is odd ? 'row-light' : 'row-dark' }}">
            <td>{{ loop.index }}</td>
            <td>{{ product.name }}</td>
            <td>${{ product.price | number_format(2) }}</td>
        </tr>
    {% endfor %}
    </tbody>
</table>
```

Empty Lists

Handle empty lists with `{% else %}`:

```
{% for product in products %}
    <div class="product-card">
        <h3>{{ product.name }}</h3>
    </div>
{% else %}
    <p>No products found.</p>
{% endfor %}
```

If `products` is empty (or not defined), the `else` block renders instead.

Looping Over Key-Value Pairs

```
{% for key, value in metadata %}
    <dt>{{ key }}</dt>
    <dd>{{ value }}</dd>
{% endfor %}
```

6. Conditionals

if / elseif / else

```
{% if user.role == "admin" %}
    <a href="/admin">Admin Panel</a>
{% elseif user.role == "editor" %}
    <a href="/editor">Editor Dashboard</a>
{% else %}
    <a href="/profile">My Profile</a>
{% endif %}
```

Ternary Operator

For inline conditionals:

```
<span class="{{ is_active ? 'text-green' : 'text-gray' }}">
    {{ is_active ? 'Active' : 'Inactive' }}
</span>
```

Testing for Existence

Use `is defined` to check if a variable exists:

```
{% if error_message is defined %}
    <div class="alert alert-danger">{{ error_message }}</div>
{% endif %}
```

Truthiness

These values are considered false: `false`, `null`, `0`, `""` (empty string), `[]` (empty array). Everything else is true.

```
{% if items %}
    <p>{{ items | length }} items found.</p>
{% else %}
    <p>No items.</p>
{% endif %}
```

7. Filters

Filters transform values. Apply them with the `|` (pipe) character:

```
{{ name | upper }}
```

Here are the most commonly used filters:

Text Filters

| Filter | Input | Output | Description |

|--------|-------|--------|-------------|

| `upper` | `"hello"` | `"HELLO"` | Uppercase |

| `lower` | `"HELLO"` | `"hello"` | Lowercase |

| `capitalize` | `"hello world"` | `"Hello world"` | Capitalize first letter |

| `title` | `"hello world"` | `"Hello World"` | Capitalize each word |

| `trim` | `" hello "` | `"hello"` | Remove whitespace |

| `striptags` | `"<b>bold</b>"` | `"bold"` | Remove HTML tags |

Number Filters

| Filter | Input | Output | Description |

|--------|-------|--------|-------------|

| `number_format(2)` | `1234.5` | `"1,234.50"` | Format number |

| `round` | `3.7` | `4` | Round to nearest integer |

| `round(2)` | `3.14159` | `3.14` | Round to N decimal places |

| `abs` | `-5` | `5` | Absolute value |

Array Filters

| Filter | Input | Output | Description |

|--------|-------|--------|-------------|

| `length` | `[1,2,3]` | `3` | Count items |

| `join(", ")` | `["a","b","c"]` | `"a, b, c"` | Join with separator |

| `first` | `[1,2,3]` | `1` | First item |

| `last` | `[1,2,3]` | `3` | Last item |

| `reverse` | `[1,2,3]` | `[3,2,1]` | Reverse order |

| `sort` | `[3,1,2]` | `[1,2,3]` | Sort ascending |

| `slice(1, 2)` | `[1,2,3,4]` | `[2,3]` | Slice from offset, length |

Date Filter

```
<p>Published: {{ created_at | date("F j, Y") }}</p>
<p>Time: {{ created_at | date("H:i") }}</p>
```

With a PHP timestamp or date string as input:

• `date("F j, Y")` outputs `"March 22, 2026"`

• `date("H:i")` outputs `"14:30"`

• `date("Y-m-d")` outputs `"2026-03-22"`

The `default` Filter

Provide a fallback value when a variable is null or undefined:

```
<p>{{ subtitle | default("No subtitle") }}</p>
<p>{{ user.nickname | default(user.name) | default("Anonymous") }}</p>
```

The `escape` and `raw` Filters

By default, all `{{ }}` output is auto-escaped for HTML safety. This prevents XSS attacks.

```
{{ user_input }}
{# If user_input is "<script>alert('xss')</script>", outputs:
   &lt;script&gt;alert('xss')&lt;/script&gt; #}
```

If you trust the content and need raw HTML:

```
{{ trusted_html | raw }}
```

Use `raw` sparingly. Only apply it to content you fully control, never to user input.

Chaining Filters

Filters chain left to right:

```
{{ name | trim | lower | capitalize }}
{# "  ALICE SMITH  " -> "Alice smith" #}
```

8. Macros

Macros are reusable template functions. Think of them as components you define once and call many times.

Defining a Macro

Create `src/templates/macros.twig`:

```
{% macro button(text, url, style) %}
    <a href="{{ url | default('#') }}" class="btn btn-{{ style | default('primary') }}">
        {{ text }}
    </a>
{% endmacro %}

{% macro alert(message, type) %}
    <div class="alert alert-{{ type | default('info') }}">
        {{ message }}
    </div>
{% endmacro %}

{% macro input(name, label, type, value) %}
    <div class="form-group">
        <label for="{{ name }}">{{ label | default(name | capitalize) }}</label>
        <input type="{{ type | default('text') }}" id="{{ name }}" name="{{ name }}" value="{{ value | defaul
    </div>
{% endmacro %}
```

Using Macros

Import macros in your template:

```
{% from "macros.twig" import button, alert, input %}

{% extends "base.twig" %}

{% block content %}
    {{ alert("Your profile has been updated.", "success") }}

    <form method="POST" action="/profile">
        {{ input("name", "Full Name", "text", user.name) }}
        {{ input("email", "Email Address", "email", user.email) }}
        {{ input("phone", "Phone Number", "tel", user.phone) }}

        {{ button("Save Changes", "", "primary") }}
        {{ button("Cancel", "/dashboard", "secondary") }}
    </form>
{% endblock %}
```

Expected output (simplified):

```
<div class="alert alert-success">
    Your profile has been updated.
</div>

<form method="POST" action="/profile">
    <div class="form-group">
        <label for="name">Full Name</label>
        <input type="text" id="name" name="name" value="Alice">
    </div>
    <div class="form-group">
        <label for="email">Email Address</label>
        <input type="email" id="email" name="email" value="alice@example.com">
    </div>
    <div class="form-group">
        <label for="phone">Phone Number</label>
        <input type="tel" id="phone" name="phone" value="">
    </div>

    <a href="#" class="btn btn-primary">Save Changes</a>
    <a href="/dashboard" class="btn btn-secondary">Cancel</a>
```

```
</form>
```

9. Special Tags

{% raw %} -- Literal Output

When you need to output literal `{{ }}` or `{% %}` (for example, in a Vue.js or Angular template):

```
{% raw %}
    <div id="app">
        {{ message }}
    </div>
{% endraw %}
```

This outputs the literal text `{{ message }}` without trying to process it as a Frond expression.

{% spaceless %} -- Remove Whitespace

Remove whitespace between HTML tags:

```
{% spaceless %}
    <div>
        <span>Hello</span>
    </div>
{% endspaceless %}
```

Output:

```
<div><span>Hello</span></div>
```

Useful for inline elements where whitespace creates unwanted gaps.

{% autoescape %} -- Control Escaping

Override the auto-escaping behavior for a block:

```
{% autoescape false %}
    {{ trusted_html }}
{% endautoescape %}
```

Everything inside the block outputs without HTML escaping. Equivalent to using `| raw` on every variable, but more convenient for large blocks of trusted content.

Comments

Template comments are invisible in the output:

```
{# This comment will not appear in the HTML output #}

{#
    Multi-line comments work too.
    Use them to document template logic.
#}
```

10. tina4css Integration

Every Tina4 project includes `tina4.css`, a built-in CSS utility framework. It is available at `/css/tina4.css` and provides layout, typography, and common UI patterns without external dependencies.

Include it in your base template:

```
<link rel="stylesheet" href="/css/tina4.css">
```

Layout Classes

```
<div class="container">
    <div class="row">
        <div class="col-6">Left half</div>
        <div class="col-6">Right half</div>
    </div>
</div>
```

Common Components

```
<!-- Buttons -->
<button class="btn btn-primary">Primary</button>
<button class="btn btn-secondary">Secondary</button>
<button class="btn btn-danger">Danger</button>

<!-- Cards -->
<div class="card">
    <div class="card-header">Title</div>
    <div class="card-body">Content here</div>
    <div class="card-footer">Footer</div>
</div>

<!-- Alerts -->
<div class="alert alert-success">Operation completed.</div>
<div class="alert alert-danger">Something went wrong.</div>
<div class="alert alert-warning">Please review your input.</div>

<!-- Forms -->
<div class="form-group">
    <label for="name">Name</label>
    <input type="text" id="name" class="form-control">
</div>
```

Utility Classes

```
<p class="text-center">Centered text</p>
<p class="text-right">Right-aligned text</p>
<div class="mt-4">Margin top</div>
<div class="p-3">Padding all around</div>
<span class="text-muted">Gray text</span>
<span class="text-primary">Primary color text</span>
```

You do not need Bootstrap or Tailwind. If you prefer those, you can replace `tina4.css` with your preferred framework -- just change the `<link>` tag.

11. Exercise: Build a Product Catalog Page

Build a product catalog page with a base layout, product cards, category filters, and a reusable card macro.

Requirements

1. Create a base layout at `src/templates/catalog-base.twig` with blocks for `title`, `content`, and `scripts`

2. Create a macro file at `src/templates/catalog-macros.twig` with:

• A `productCard(product)` macro that renders a styled card with name, category, price, stock status, and optional featured badge

• A `categoryFilter(categories, active)` macro that renders filter buttons

3. Create a page template at `src/templates/catalog.twig` that:

• Extends the base layout

• Uses the macros

• Shows a heading with the total product count

• Shows category filter buttons (All, and one per unique category)

• Shows product cards in a grid

• Shows featured products with a distinct style

• Handles the case when no products match the filter

4. Create a route at `GET /catalog` that accepts an optional `?category=` filter

Data

Use this product list in your route handler:

```
$products = [
    ["name" => "Espresso Machine", "category" => "Kitchen", "price" => 299.99, "in_stock" => true, "featured"
    ["name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "in_stock" => true, "featured" => false
    ["name" => "Standing Desk", "category" => "Office", "price" => 549.99, "in_stock" => true, "featured" => 
    ["name" => "Blender", "category" => "Kitchen", "price" => 89.99, "in_stock" => false, "featured" => false
    ["name" => "Running Shoes", "category" => "Fitness", "price" => 119.99, "in_stock" => true, "featured" =>
    ["name" => "Desk Lamp", "category" => "Office", "price" => 39.99, "in_stock" => true, "featured" => true],
    ["name" => "Cast Iron Skillet", "category" => "Kitchen", "price" => 44.99, "in_stock" => true, "featured"
];
```

Test with:

```
http://localhost:7145/catalog
http://localhost:7145/catalog?category=Kitchen
http://localhost:7145/catalog?category=Fitness
```

12. Solution

Create `src/templates/catalog-base.twig`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Product Catalog{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; margin: 0; pad
        .container { max-width: 1000px; margin: 0 auto; padding: 20px; }
        .header { background: #2c3e50; color: white; padding: 20px; margin-bottom: 24px; }
        .header h1 { margin: 0; }
        .header p { margin: 4px 0 0; opacity: 0.8; }
        .filters { margin-bottom: 20px; }
        .filter-btn { display: inline-block; padding: 6px 14px; margin: 0 6px 6px 0; border-radius: 20px; tex
```

```
        .filter-btn.active { background: #2c3e50; color: white; border-color: #2c3e50; }
        .product-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(280px, 1fr)); gap: 16p
        .product-card { background: white; border: 2px solid #e9ecef; border-radius: 8px; padding: 16px; trans
        .product-card:hover { border-color: #adb5bd; }
        .product-card.featured { border-color: #f39c12; background: #fef9e7; }
        .product-name { font-size: 1.1em; font-weight: 600; margin: 0 0 4px; }
        .product-category { font-size: 0.85em; color: #6c757d; margin: 0 0 8px; }
        .product-price { font-size: 1.2em; font-weight: bold; color: #27ae60; }
        .badge { display: inline-block; padding: 2px 8px; border-radius: 4px; font-size: 0.75em; font-weight:
        .badge-featured { background: #f39c12; color: white; }
// ...
```

Create `src/templates/catalog-macros.twig`:

```
{% macro productCard(product) %}
    <div class="product-card{{ product.featured ? ' featured' : '' }}">
        <p class="product-name">
            {{ product.name }}
            {% if product.featured %}
                <span class="badge badge-featured">Featured</span>
            {% endif %}
        </p>
        <p class="product-category">{{ product.category }}</p>
        <p class="product-price">
            ${{ product.price | number_format(2) }}
            {% if product.in_stock %}
                <span class="badge badge-stock">In Stock</span>
            {% else %}
                <span class="badge badge-nostock">Out of Stock</span>
            {% endif %}
        </p>
    </div>
{% endmacro %}

{% macro categoryFilter(categories, active) %}
    <div class="filters">
        <a href="/catalog" class="filter-btn{{ active is not defined or active == '' ? ' active' : '' }}">All
        {% for cat in categories %}
            <a href="/catalog?category={{ cat }}" class="filter-btn{{ active == cat ? ' active' : '' }}">{{ ca
// ...
```

Create `src/templates/catalog.twig`:

```
{% extends "catalog-base.twig" %}

{% from "catalog-macros.twig" import productCard, categoryFilter %}

{% block title %}{{ active_category | default("All") }} Products - Catalog{% endblock %}

{% block content %}
    <div class="header">
        <h1>Product Catalog</h1>
        <p>{{ products | length }} product{{ products | length != 1 ? 's' : '' }}{% if active_category %} in
    </div>

    <div class="container">
        {{ categoryFilter(categories, active_category) }}

        {% if products | length > 0 %}
            <div class="product-grid">
                {% for product in products %}
                    {{ productCard(product) }}
                {% endfor %}
            </div>
```

```
        {% else %}
            <div class="empty-state">
                <h2>No products found</h2>
                <p>Try a different category or <a href="/catalog">view all products</a>.</p>
// ...
```

Create `src/routes/catalog.php`:

```
<?php
use Tina4\Route;

Route::get("/catalog", function ($request, $response) {
    $allProducts = [
        ["name" => "Espresso Machine", "category" => "Kitchen", "price" => 299.99, "in_stock" => true, "featu
        ["name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "in_stock" => true, "featured" => fa
        ["name" => "Standing Desk", "category" => "Office", "price" => 549.99, "in_stock" => true, "featured"
        ["name" => "Blender", "category" => "Kitchen", "price" => 89.99, "in_stock" => false, "featured" => fa
        ["name" => "Running Shoes", "category" => "Fitness", "price" => 119.99, "in_stock" => true, "featured"
        ["name" => "Desk Lamp", "category" => "Office", "price" => 39.99, "in_stock" => true, "featured" => tr
        ["name" => "Cast Iron Skillet", "category" => "Kitchen", "price" => 44.99, "in_stock" => true, "featu
    ];

    // Get unique categories
    $categories = array_unique(array_column($allProducts, "category"));
    sort($categories);

    // Filter by category if specified
    $activeCategory = $request->query["category"] ?? "";
    if (!empty($activeCategory)) {
        $products = array_values(array_filter(
            $allProducts,
            fn($p) => strtolower($p["category"]) === strtolower($activeCategory)
        ));
// ...
```

Expected browser output for `/catalog`:

• A dark header with "Product Catalog" and "7 products"

• Filter buttons: All (active), Fitness, Kitchen, Office

• A grid of 7 product cards

• Three cards (Espresso Machine, Standing Desk, Desk Lamp) have a gold border and "Featured" badge

• The Blender card shows an "Out of Stock" badge in red

Expected browser output for `/catalog?category=Kitchen`:

• Header shows "3 products in Kitchen"

• The Kitchen filter button is active

• Only three cards: Espresso Machine, Blender, Cast Iron Skillet

13. Gotchas

1. `{% extends %}` Must Be the First Tag

Problem: Template inheritance does not work. The page renders without the base layout.

Cause: `{% extends "base.twig" %}` must be the very first tag in the template. If there is any text, whitespace, or even a comment before it, inheritance breaks.

Fix: Make `{% extends %}` the absolute first thing in the file. Move any `{% from %}` imports to after the extends tag.

2. Undefined Variables Show Nothing

Problem: `{{ username }}` renders as empty instead of showing an error.

Cause: Frond silently outputs nothing for undefined variables. This is by design (like Twig), but it can hide bugs.

Fix: Use the `default` filter: `{{ username | default("Guest") }}`. Or check first with `{% if username is defined %}`.

3. Auto-Escaping Prevents HTML Output

Problem: You pass HTML content like `"<strong>bold</strong>"` but it appears as literal text in the page.

Cause: Auto-escaping converts `<` to `<` and `>` to `>` for security.

Fix: If the content is trusted, use `{{ content | raw }}`. Never use `raw` on user-supplied input.

4. Variable Scope in Includes

Problem: A variable defined inside a `{% for %}` loop is not accessible after the loop ends.

Cause: Loop variables are scoped to the loop. They do not leak into the outer scope.

Fix: If you need to accumulate values, use `{% set %}` before the loop and update it inside. Or restructure your template to keep all logic within the loop.

5. Macro Arguments Are Positional

Problem: Calling `{{ button("Click", style="danger") }}` does not work as expected.

Cause: Frond macros use positional arguments, not keyword arguments. The order matters.

Fix: Pass arguments in the order they are defined: `{{ button("Click", "/url", "danger") }}`. If you have many optional arguments, consider passing a single object.

6. Template File Extension Does Not Matter

Problem: You are not sure whether to use `.html`, `.twig`, or `.tpl`.

Cause: Frond does not care about the file extension. It processes any file in `src/templates/`.

Fix: Pick one extension and be consistent. This book uses `.twig` for templates with Twig syntax and `.html` for simple HTML files. Both work identically.

7. Filters Are Not PHP Functions

Problem: You try `{{ items | count }}` or `{{ name | strtoupper }}` and get an error.

Cause: Frond filters are not PHP functions. The filter names follow Twig conventions, not PHP conventions.

Fix: Use `{{ items | length }}` instead of `count`, `{{ name | upper }}` instead of `strtoupper`, `{{ text | lower }}` instead of `strtolower`. See the filter table in section 7 for the full list.

Chapter 5: Database

1. From Arrays to Real Data

In Chapters 2 and 3, all our data lived in PHP arrays that reset every time the server restarted. That is fine for learning routing and responses, but real applications need persistent storage. This chapter covers Tina4's database layer -- raw queries, parameterised queries, transactions, schema inspection, helper methods, and migrations.

Tina4 supports five database engines: SQLite, PostgreSQL, MySQL, Microsoft SQL Server, and Firebird. The API is the same across all of them. You switch databases by changing one line in `.env`.

2. Connecting to a Database

The Default: SQLite

When you scaffold a project with `tina4 init`, Tina4 creates a SQLite database at `data/app.db`. The default `.env` includes:

```
TINA4_DEBUG=true
```

With no explicit `DATABASE_URL`, Tina4 defaults to `sqlite:///data/app.db`. That is why the health check at `/health` already shows `"database": "connected"` without any configuration.

Connection Strings for Other Databases

Set `DATABASE_URL` in `.env` to use a different engine:

```
# SQLite (explicit)
DATABASE_URL=sqlite:///data/app.db

# PostgreSQL
DATABASE_URL=postgres://localhost:5432/myapp

# MySQL
DATABASE_URL=mysql://localhost:3306/myapp

# Microsoft SQL Server
DATABASE_URL=mssql://localhost:1433/myapp

# Firebird
DATABASE_URL=firebird://localhost:3050/path/to/database.fdb
```

Separate Credentials

If you prefer to keep credentials out of the connection string (recommended for production), use separate environment variables:

```
DATABASE_URL=postgres://localhost:5432/myapp
DATABASE_USERNAME=myuser
DATABASE_PASSWORD=secretpassword
```

Tina4 merges these with the connection string at startup. The credentials in the separate variables take precedence over any embedded in the URL.

Verifying the Connection

After updating `.env`, restart the server and check:

```
curl http://localhost:7145/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 3,
  "version": "3.0.0",
  "framework": "tina4-php"
}
```

If the database is not reachable, you will see `"database": "disconnected"` with an error message.

3. Getting the Database Object

In your route handlers, access the database via the global `Tina4\Database` class:

```
<?php
use Tina4\Route;
use Tina4\Database;

Route::get("/api/test-db", function ($request, $response) {
    $db = Database::getConnection();

    $result = $db->fetch("SELECT 1 + 1 AS answer");

    return $response->json($result);
});
```

```
curl http://localhost:7145/api/test-db
```

```
{"answer": 2}
```

`Database::getConnection()` returns the active database connection. You call methods like `fetch()`, `execute()`, and `fetchOne()` on this object.

4. Raw Queries

fetch() -- Get Multiple Rows

```
$db = Database::getConnection();

// Returns an array of associative arrays
$products = $db->fetch("SELECT * FROM products WHERE price > 50");
```

Each row is an associative array with column names as keys:

```
// $products looks like:
[
    ["id" => 1, "name" => "Keyboard", "price" => 79.99],
    ["id" => 4, "name" => "Standing Desk", "price" => 549.99]
]
```

fetchOne() -- Get a Single Row

```
$product = $db->fetchOne("SELECT * FROM products WHERE id = 1");
// Returns: ["id" => 1, "name" => "Keyboard", "price" => 79.99]
```

If no row matches, `fetchOne()` returns `null`.

execute() -- Run a Statement

For INSERT, UPDATE, DELETE, and DDL statements that do not return rows:

```
$db->execute("INSERT INTO products (name, price) VALUES ('Widget', 9.99)");
$db->execute("UPDATE products SET price = 89.99 WHERE id = 1");
$db->execute("DELETE FROM products WHERE id = 5");
$db->execute("CREATE TABLE IF NOT EXISTS logs (id INTEGER PRIMARY KEY, message TEXT, created_at TEXT)");
```

Full Example: A Simple Query Route

```
<?php
use Tina4\Route;
use Tina4\Database;

Route::get("/api/products", function ($request, $response) {
    $db = Database::getConnection();

    $products = $db->fetch("SELECT * FROM products ORDER BY name");

    return $response->json([
        "products" => $products,
        "count" => count($products)
    ]);
});
```

```
curl http://localhost:7145/api/products
```

```
{
  "products": [
    {"id": 1, "name": "Keyboard", "price": 79.99, "in_stock": 1},
    {"id": 2, "name": "Mouse", "price": 29.99, "in_stock": 1},
    {"id": 3, "name": "Monitor", "price": 399.99, "in_stock": 0}
  ],
  "count": 3
}
```

5. Parameterised Queries

Never concatenate user input into SQL strings. This is how SQL injection attacks happen:

```
// NEVER do this:
$db->fetch("SELECT * FROM products WHERE name = '" . $userInput . "'");
```

Instead, use parameterised queries. Pass parameters as the second argument:

```
$db = Database::getConnection();

// Named parameters
$product = $db->fetchOne(
    "SELECT * FROM products WHERE id = :id",
    ["id" => 42]
);

// Positional parameters
$products = $db->fetch(
    "SELECT * FROM products WHERE price BETWEEN ? AND ? ORDER BY price",
    [10.00, 100.00]
);
```

The database driver handles escaping. Your input is never part of the SQL string.

A Safe Search Endpoint

```php
<?php
use Tina4\Route;
use Tina4\Database;

Route::get("/api/products/search", function ($request, $response) {
    $db = Database::getConnection();

    $q = $request->query["q"] ?? "";
    $maxPrice = (float) ($request->query["max_price"] ?? 99999);

    if (empty($q)) {
        return $response->json(["error" => "Query parameter 'q' is required"], 400);
    }

    $products = $db->fetch(
        "SELECT * FROM products WHERE name LIKE :query AND price <= :maxPrice ORDER BY name",
        ["query" => "%" . $q . "%", "maxPrice" => $maxPrice]
    );

    return $response->json([
        "query" => $q,
        "max_price" => $maxPrice,
        "results" => $products,
        "count" => count($products)
    ]);
// ...
```

```bash
curl "http://localhost:7145/api/products/search?q=key&max_price=100"
```

```json
{
  "query": "key",
  "max_price": 100,
  "results": [
    {"id": 1, "name": "Wireless Keyboard", "price": 79.99, "in_stock": 1}
  ],
  "count": 1
}
```

6. Transactions

When you need multiple operations to succeed or fail together, use transactions:

```php
<?php
use Tina4\Route;
use Tina4\Database;

Route::post("/api/orders", function ($request, $response) {
    $db = Database::getConnection();
    $body = $request->body;

    try {
        $db->startTransaction();

        // Create the order
        $db->execute(
            "INSERT INTO orders (customer_id, total, status) VALUES (:customerId, :total, 'pending')",
            ["customerId" => $body["customer_id"], "total" => $body["total"]]
        );

        // Get the new order ID
```

```
        $order = $db->fetchOne("SELECT last_insert_rowid() AS id");
        $orderId = $order["id"];

        // Create order items
        foreach ($body["items"] as $item) {
            $db->execute(
                "INSERT INTO order_items (order_id, product_id, quantity, price) VALUES (:orderId, :productId,
// ...
```

If any step fails, `rollback()` undoes everything. The database is never left in a half-finished state.

Important: You must call `commit()` to save the changes. If you forget, the transaction will be rolled back when the connection closes.

7. Schema Inspection

Tina4 provides methods to inspect your database structure at runtime:

getTables()

```
$db = Database::getConnection();
$tables = $db->getTables();
```

Returns an array of table names:

```
["orders", "order_items", "products", "users"]
```

getColumns()

```
$columns = $db->getColumns("products");
```

Returns an array of column definitions:

```
[
    ["name" => "id", "type" => "INTEGER", "nullable" => false, "primary" => true],
    ["name" => "name", "type" => "TEXT", "nullable" => false, "primary" => false],
    ["name" => "price", "type" => "REAL", "nullable" => true, "primary" => false],
    ["name" => "in_stock", "type" => "INTEGER", "nullable" => true, "primary" => false]
]
```

tableExists()

```
if ($db->tableExists("products")) {
    // Table exists, safe to query
}
```

A Schema Info Endpoint

```
<?php
use Tina4\Route;
use Tina4\Database;

Route::get("/api/schema", function ($request, $response) {
    $db = Database::getConnection();
    $tables = $db->getTables();

    $schema = [];
    foreach ($tables as $table) {
        $schema[$table] = $db->getColumns($table);
    }

    return $response->json(["tables" => $schema]);
```

```
});
```

8. Batch Operations with executeMany()

Insert or update many rows efficiently:

```
$db = Database::getConnection();

$products = [
    ["name" => "Widget A", "price" => 9.99],
    ["name" => "Widget B", "price" => 14.99],
    ["name" => "Widget C", "price" => 19.99],
    ["name" => "Widget D", "price" => 24.99]
];

$db->executeMany(
    "INSERT INTO products (name, price) VALUES (:name, :price)",
    $products
);
```

`executeMany()` prepares the statement once and executes it for each item in the array. This is significantly faster than calling `execute()` in a loop because the SQL only needs to be parsed once.

9. Helper Methods: insert(), update(), delete()

Tina4 provides shorthand methods so you do not have to write SQL for simple operations.

insert()

```
$db = Database::getConnection();

// Insert a single row
$db->insert("products", [
    "name" => "Wireless Mouse",
    "price" => 34.99,
    "in_stock" => 1
]);

// Insert multiple rows
$db->insert("products", [
    ["name" => "USB Cable", "price" => 9.99, "in_stock" => 1],
    ["name" => "HDMI Cable", "price" => 14.99, "in_stock" => 1],
    ["name" => "DisplayPort Cable", "price" => 19.99, "in_stock" => 0]
]);
```

update()

```
// Update rows matching a filter
$db->update("products", ["price" => 39.99, "in_stock" => 1], "id = :id", ["id" => 7]);
```

The third argument is the WHERE clause, and the fourth is the parameters for it.

delete()

```
// Delete rows matching a filter
$db->delete("products", "id = :id", ["id" => 7]);
```

These helper methods generate the SQL for you. They are convenient for simple CRUD operations but do not replace raw queries for complex joins, subqueries, or aggregations.

10. Migrations

Migrations are versioned SQL scripts that evolve your database schema over time. Instead of manually running `CREATE TABLE` statements, you write migration files and Tina4 applies them in order.

Creating a Migration

Use the CLI:

```
tina4 migrate:create create_products_table
```

```
Created migration: src/migrations/20260322143000_create_products_table.sql
```

The filename starts with a timestamp so migrations always run in chronological order.

Edit the generated file `src/migrations/20260322143000_create_products_table.sql`:

```
-- UP
CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    category TEXT NOT NULL DEFAULT 'Uncategorized',
    price REAL NOT NULL DEFAULT 0.00,
    in_stock INTEGER NOT NULL DEFAULT 1,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS products;
```

The `-- UP` section runs when applying the migration. The `-- DOWN` section runs when rolling back.

Running Migrations

```
tina4 migrate
```

```
Running migrations...
  [APPLIED] 20260322143000_create_products_table.sql
Migrations complete. 1 applied.
```

Checking Migration Status

```
tina4 migrate:status
```

```
Migration                                      Status     Applied At
---------                                      ------     ----------
20260322143000_create_products_table.sql        applied    2026-03-22 14:30:00
20260322150000_create_orders_table.sql          pending    -
```

Rolling Back

```
tina4 migrate:rollback
```

```
Rolling back last migration...
  [ROLLED BACK] 20260322150000_create_orders_table.sql
Rollback complete. 1 rolled back.
```

This runs the `-- DOWN` section of the most recently applied migration.

A Real Migration Sequence

Here is what a typical project's migrations look like:

```
src/migrations/
■■■ 20260322143000_create_products_table.sql
■■■ 20260322143100_create_users_table.sql
■■■ 20260322143200_create_orders_table.sql
■■■ 20260322143300_create_order_items_table.sql
■■■ 20260323091500_add_email_index_to_users.sql
```

The last migration might look like:

```
-- UP
CREATE INDEX idx_users_email ON users (email);

-- DOWN
DROP INDEX IF EXISTS idx_users_email;
```

Migrations are applied in filename order. Each migration runs only once -- Tina4 tracks which ones have been applied in a `_migrations` table.

11. Query Caching

For read-heavy applications, enable query caching:

```
TINA4_DB_CACHE=true
```

When enabled, Tina4 caches the results of `fetch()` and `fetchOne()` calls. Identical queries with identical parameters return cached results instead of hitting the database again.

The cache is automatically invalidated when you call `execute()`, `insert()`, `update()`, or `delete()` on the same table.

You can also control caching per-query:

```
// Force a fresh query (bypass cache)
$products = $db->fetch("SELECT * FROM products", [], false); // third arg = use cache

// Clear the entire cache
$db->clearCache();
```

12. Exercise: Build a Notes App

Build a notes application backed by SQLite. Create the database table via a migration and build a full CRUD API.

Requirements

1. Create a migration that creates a `notes` table with columns:

• `id` -- integer, primary key, auto-increment

• `title` -- text, not null

• `content` -- text, not null

• `tag` -- text, default "general"

• `created_at` -- text, default current timestamp

• `updated_at` -- text, default current timestamp

2. Build these API endpoints:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/notes` | List all notes. Support `?tag=` and `?search=` filters. |

| `GET` | `/api/notes/{id:int}` | Get a single note. 404 if not found. |

| `POST` | `/api/notes` | Create a note. Validate title and content are not empty. |

| `PUT` | `/api/notes/{id:int}` | Update a note. 404 if not found. |

| `DELETE` | `/api/notes/{id:int}` | Delete a note. 204 on success, 404 if not found. |

Test with:

```
# Create
curl -X POST http://localhost:7145/api/notes \
  -H "Content-Type: application/json" \
  -d '{"title": "Shopping List", "content": "Milk, eggs, bread", "tag": "personal"}'

# List all
curl http://localhost:7145/api/notes

# Search
curl "http://localhost:7145/api/notes?search=shopping"

# Filter by tag
curl "http://localhost:7145/api/notes?tag=personal"

# Get one
curl http://localhost:7145/api/notes/1

# Update
curl -X PUT http://localhost:7145/api/notes/1 \
  -H "Content-Type: application/json" \
  -d '{"title": "Updated Shopping List", "content": "Milk, eggs, bread, butter"}'

# Delete
curl -X DELETE http://localhost:7145/api/notes/1
```

13. Solution

Migration

Create `src/migrations/20260322143000_create_notes_table.sql`:

```
-- UP
CREATE TABLE notes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    content TEXT NOT NULL,
    tag TEXT NOT NULL DEFAULT 'general',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS notes;
```

Run the migration:

```
tina4 migrate
```

```
Running migrations...
  [APPLIED] 20260322143000_create_notes_table.sql
Migrations complete. 1 applied.
```

Routes

Create `src/routes/notes.php`:

```
<?php
use Tina4\Route;
use Tina4\Database;

// List all notes with optional filters
Route::get("/api/notes", function ($request, $response) {
    $db = Database::getConnection();

    $tag = $request->query["tag"] ?? "";
    $search = $request->query["search"] ?? "";

    $sql = "SELECT * FROM notes";
    $params = [];
    $conditions = [];

    if (!empty($tag)) {
        $conditions[] = "tag = :tag";
        $params["tag"] = $tag;
    }

    if (!empty($search)) {
        $conditions[] = "(title LIKE :search OR content LIKE :search)";
        $params["search"] = "%" . $search . "%";
    }

// ...
```

Expected output for create:

```
{
  "id": 1,
  "title": "Shopping List",
  "content": "Milk, eggs, bread",
  "tag": "personal",
  "created_at": "2026-03-22 14:30:00",
  "updated_at": "2026-03-22 14:30:00"
}
```

(Status: `201 Created`)

Expected output for list:

```
{
  "notes": [
    {
      "id": 1,
      "title": "Shopping List",
      "content": "Milk, eggs, bread",
      "tag": "personal",
      "created_at": "2026-03-22 14:30:00",
      "updated_at": "2026-03-22 14:30:00"
    }
  ],
```

```
  "count": 1
}
```

Expected output for search:

```
{
  "notes": [
    {
      "id": 1,
      "title": "Shopping List",
      "content": "Milk, eggs, bread",
      "tag": "personal",
      "created_at": "2026-03-22 14:30:00",
      "updated_at": "2026-03-22 14:30:00"
    }
  ],
  "count": 1
}
```

Expected output for validation error:

```
{"errors": ["Title is required", "Content is required"]}
```

(Status: `400 Bad Request`)

14. Gotchas

1. Forgetting commit()

Problem: You call `startTransaction()`, run your queries, but the changes disappear on the next request.

Cause: Without `commit()`, the transaction is rolled back when the connection closes.

Fix: Always call `$db->commit()` after your transaction succeeds. Use a try/catch block with `$db->rollback()` in the catch.

2. Connection String Formats

Problem: The database will not connect and you see a cryptic error about the connection string.

Cause: Each database engine expects a specific URL format. A common mistake is using `mysql://user:pass@host/db` when the engine expects the port.

Fix: Always include the port: `mysql://localhost:3306/mydb`. Here are the default ports:

| Engine | Default Port |

|--------|-------------|

| PostgreSQL | 5432 |

| MySQL | 3306 |

| MSSQL | 1433 |

| Firebird | 3050 |

| SQLite | (file path, no port) |

3. SQLite File Paths

Problem: SQLite creates a new empty database instead of using the existing one.

Cause: The path in `DATABASE_URL` is relative and resolves to the wrong directory, or you used `sqlite://` (two slashes) instead of `sqlite:///` (three slashes).

Fix: Use three slashes for a relative path: `sqlite:///data/app.db`. For an absolute path, use four slashes: `sqlite:////var/data/app.db`. The third slash separates the scheme from the path; the fourth starts the absolute path.

4. Parameterised Queries with LIKE

Problem: `WHERE name LIKE :q` with `["q" => "%search%"]` works, but `WHERE name LIKE '%:q%'` does not.

Cause: Parameters inside quotes are treated as literal text, not as placeholders.

Fix: Include the `%` wildcards in the parameter value, not in the SQL: `["q" => "%" . $search . "%"]`. The SQL should be `WHERE name LIKE :q`.

5. Boolean Values in SQLite

Problem: You insert `true` or `false` but the database stores `1` or `0`. When you read it back, you get integers, not booleans.

Cause: SQLite does not have a native boolean type. It stores booleans as integers.

Fix: Cast in your PHP code: `"in_stock" => (bool) $row["in_stock"]`. Or accept that `1` and `0` work as truthy/falsy in PHP.

6. Migration Already Applied

Problem: You edited a migration file and ran `tina4 migrate` again, but nothing changed.

Cause: Tina4 tracks applied migrations by filename. Once applied, a migration will not run again even if you change its contents.

Fix: Create a new migration for schema changes. Do not edit applied migrations. If you are in early development and want to start fresh, use `tina4 migrate:rollback` to undo and then `tina4 migrate` to reapply.

7. fetch() Returns Empty Array, Not Null

Problem: You check `if ($result === null)` but it never matches, even when the table is empty.

Cause: `fetch()` always returns an array. An empty result is `[]`, not `null`. Only `fetchOne()` returns `null` when no row matches.

Fix: Check with `if (empty($result))` or `if (count($result) === 0)`.

Chapter 6: ORM

1. From SQL to Objects

In Chapter 5 you wrote raw SQL for every operation. That works, but it gets tedious. You end up writing the same `INSERT INTO products (name, price, ...) VALUES (:name, :price, ...)` patterns over and over. The ORM (Object-Relational Mapper) lets you work with PHP classes instead. You define a class, map it to a table, and call methods like `save()`, `load()`, and `delete()`.

Tina4's ORM is minimal by design. It does not try to hide SQL completely -- it gives you convenient methods for common operations and stays out of the way when you need raw queries.

2. Defining a Model

ORM models live in `src/orm/`. Every `.php` file in that directory is auto-loaded, just like route files.

Create `src/orm/Product.php`:

```
<?php
use Tina4\ORM;

class Product extends ORM
{
    public int $id;
    public string $name;
    public string $category = "Uncategorized";
    public float $price = 0.00;
    public bool $inStock = true;
    public string $createdAt;
    public string $updatedAt;

    // Map to the "products" table
    public string $tableName = "products";

    // Primary key field
    public string $primaryKey = "id";
}
```

That is a complete model. Let us break it down:

• **Extends `ORM`** -- This gives you `save()`, `load()`, `delete()`, `select()`, and other methods.

• **Public properties** -- Each property maps to a database column. The property name is `camelCase` and the column name is `snake_case`. Tina4 converts between them automatically: `inStock` maps to `in_stock`, `createdAt` maps to `created_at`.

• **`$tableName`** -- The database table this model maps to. If you omit it, Tina4 infers it from the class name: `Product` becomes `products`, `OrderItem` becomes `order_items`.

• **`$primaryKey`** -- The primary key column. Defaults to `"id"`.

• **Default values** -- Properties with defaults (like `$category = "Uncategorized"`) are used when creating new records without specifying those fields.

3. Field Types

Use PHP type declarations on your properties. Tina4 uses these types when generating DDL and validating data:

| PHP Type | Database Type (SQLite) | Database Type (PostgreSQL) | Notes |

|----------|----------|----------|-------|

| `int` | INTEGER | INTEGER | Whole numbers |

| `string` | TEXT | VARCHAR(255) | Text fields |

| `float` | REAL | DOUBLE PRECISION | Decimal numbers |

| `bool` | INTEGER | BOOLEAN | SQLite stores as 0/1 |

| `?string` | TEXT (nullable) | VARCHAR(255) NULL | Nullable with `?` prefix |

Nullable Fields

Use PHP's nullable type syntax:

```
public ?string $description = null;
public ?float $discount = null;
```

A nullable property allows `null` values in the database column.

Primary Keys and Auto-Increment

By default, Tina4 treats the `$primaryKey` field as auto-incrementing. When you call `save()` on a new object (where the primary key is not set), the database generates the ID:

```
$product = new Product();
$product->name = "Widget";
$product->price = 9.99;
$product->save();

echo $product->id; // Auto-generated: 1, 2, 3, ...
```

4. Creating and Saving Records

save() -- Insert or Update

The `save()` method inserts a new record or updates an existing one, depending on whether the primary key is set:

```
<?php
use Tina4\Route;

Route::post("/api/products", function ($request, $response) {
    $body = $request->body;

    $product = new Product();
    $product->name = $body["name"];
    $product->category = $body["category"] ?? "Uncategorized";
    $product->price = (float) ($body["price"] ?? 0);
    $product->inStock = (bool) ($body["in_stock"] ?? true);
    $product->save();

    return $response->json($product->toDict(), 201);
```

```
});
```

```
curl -X POST http://localhost:7145/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Wireless Keyboard", "category": "Electronics", "price": 79.99}'
```

```
{
  "id": 1,
  "name": "Wireless Keyboard",
  "category": "Electronics",
  "price": 79.99,
  "in_stock": true,
  "created_at": "2026-03-22 14:30:00",
  "updated_at": "2026-03-22 14:30:00"
}
```

Updating an Existing Record

When `id` is already set, `save()` performs an UPDATE:

```
Route::put("/api/products/{id:int}", function ($request, $response) {
    $product = new Product();
    $product->load($request->params["id"]);

    if (empty($product->id)) {
        return $response->json(["error" => "Product not found"], 404);
    }

    $body = $request->body;
    $product->name = $body["name"] ?? $product->name;
    $product->price = (float) ($body["price"] ?? $product->price);
    $product->category = $body["category"] ?? $product->category;
    $product->save();

    return $response->json($product->toDict());
});
```

5. Loading Records

load() -- Get by Primary Key

```
$product = new Product();
$product->load(42);

if (empty($product->id)) {
    // Product with ID 42 not found
}
```

`load()` populates the object's properties from the database row matching the primary key. If no row matches, the properties remain at their default values (or unset).

A Simple Get Endpoint

```
Route::get("/api/products/{id:int}", function ($request, $response) {
    $product = new Product();
    $product->load($request->params["id"]);

    if (empty($product->id)) {
        return $response->json(["error" => "Product not found"], 404);
    }

    return $response->json($product->toDict());
```

```
});
```

```
curl http://localhost:7145/api/products/1
```

```
{
  "id": 1,
  "name": "Wireless Keyboard",
  "category": "Electronics",
  "price": 79.99,
  "in_stock": true,
  "created_at": "2026-03-22 14:30:00",
  "updated_at": "2026-03-22 14:30:00"
}
```

6. Deleting Records

delete()

```
Route::delete("/api/products/{id:int}", function ($request, $response) {
    $product = new Product();
    $product->load($request->params["id"]);

    if (empty($product->id)) {
        return $response->json(["error" => "Product not found"], 404);
    }

    $product->delete();

    return $response->json(null, 204);
});
```

`delete()` removes the row from the database. The object still exists in memory but the database row is gone.

7. Querying with select()

The `select()` method lets you find records with filters, ordering, and pagination:

Basic Select

```
$product = new Product();
$products = $product->select("*");
```

Returns an array of Product objects with all records.

Filtering

```
$product = new Product();

// Simple filter
$electronics = $product->select("*", "category = :category", ["category" => "Electronics"]);

// Multiple conditions
$affordable = $product->select("*", "price < :maxPrice AND in_stock = :inStock", [
    "maxPrice" => 100,
    "inStock" => 1
]);
```

Ordering

```
$product = new Product();
$sorted = $product->select("*", "", [], "price DESC");
```

The fourth argument is the ORDER BY clause.

Pagination

```
$product = new Product();

$page = 1;
$perPage = 10;
$offset = ($page - 1) * $perPage;

$products = $product->select("*", "", [], "name ASC", $perPage, $offset);
```

The fifth argument is LIMIT, the sixth is OFFSET.

A Full List Endpoint with Filters

```
<?php
use Tina4\Route;

Route::get("/api/products", function ($request, $response) {
    $product = new Product();

    $category = $request->query["category"] ?? "";
    $minPrice = (float) ($request->query["min_price"] ?? 0);
    $maxPrice = (float) ($request->query["max_price"] ?? 999999);
    $page = (int) ($request->query["page"] ?? 1);
    $perPage = (int) ($request->query["per_page"] ?? 20);
    $sort = $request->query["sort"] ?? "name";
    $order = strtoupper($request->query["order"] ?? "ASC");

    // Build filter
    $conditions = [];
    $params = [];

    if (!empty($category)) {
        $conditions[] = "category = :category";
        $params["category"] = $category;
    }

    $conditions[] = "price >= :minPrice AND price <= :maxPrice";
    $params["minPrice"] = $minPrice;
// ...
```

```
curl "http://localhost:7145/api/products?category=Electronics&sort=price&order=DESC&page=1&per_page=5"
```

```
{
  "products": [
    {"id": 4, "name": "Standing Desk", "category": "Electronics", "price": 549.99, "in_stock": true},
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock": true}
  ],
  "page": 1,
  "per_page": 5,
  "count": 2
}
```

8. Creating Tables from Models

Instead of writing a migration manually, you can generate the table from your model:

```
$product = new Product();
$product->createTable();
```

This reads the properties, types, and defaults from the model class and generates the appropriate `CREATE TABLE` statement for your database engine.

You can also use the CLI:

```
tina4 orm:create-table Product
```

```
Created table "products" with 7 columns.
```

This is convenient during early development. For production, use migrations (Chapter 5) so schema changes are versioned and reversible.

9. Relationships

hasMany -- One-to-Many

A user has many posts:

Create `src/orm/User.php`:

```
<?php
use Tina4\ORM;

class User extends ORM
{
    public int $id;
    public string $name;
    public string $email;
    public string $createdAt;

    public string $tableName = "users";
    public string $primaryKey = "id";

    public function posts(): array
    {
        return $this->hasMany(Post::class, "user_id");
    }
}
```

Create `src/orm/Post.php`:

```
<?php
use Tina4\ORM;

class Post extends ORM
{
    public int $id;
    public int $userId;
    public string $title;
    public string $body;
    public string $createdAt;

    public string $tableName = "posts";
    public string $primaryKey = "id";

    public function user(): ?User
    {
        return $this->belongsTo(User::class, "user_id");
    }
```

```
    public function comments(): array
    {
        return $this->hasMany(Comment::class, "post_id");
    }
}
```

The second argument to `hasMany()` is the foreign key column on the related table. `$this->hasMany(Post::class, "user_id")` means: find all rows in `posts` where `user_id` equals this user's ID.

hasOne -- One-to-One

```
public function profile(): ?Profile
{
    return $this->hasOne(Profile::class, "user_id");
}
```

`hasOne()` works like `hasMany()` but returns a single object instead of an array.

belongsTo -- Inverse Relationship

The inverse of `hasMany`. A post belongs to a user:

```
public function user(): ?User
{
    return $this->belongsTo(User::class, "user_id");
}
```

`belongsTo(User::class, "user_id")` means: load the User where `users.id` equals `this->user_id`.

Using Relationships

```
Route::get("/api/users/{id:int}", function ($request, $response) {
    $user = new User();
    $user->load($request->params["id"]);

    if (empty($user->id)) {
        return $response->json(["error" => "User not found"], 404);
    }

    $posts = $user->posts();

    return $response->json([
        "user" => $user->toDict(),
        "posts" => array_map(fn($p) => $p->toDict(), $posts),
        "post_count" => count($posts)
    ]);
});
```

```
curl http://localhost:7145/api/users/1
```

```
{
  "user": {"id": 1, "name": "Alice", "email": "alice@example.com"},
  "posts": [
    {"id": 1, "user_id": 1, "title": "First Post", "body": "Hello world!"},
    {"id": 3, "user_id": 1, "title": "Second Post", "body": "Another one."}
  ],
  "post_count": 2
}
```

10. Eager Loading

Calling relationship methods inside a loop creates the N+1 query problem. If you load 100 users and then call `$user->posts()` for each one, that is 101 queries (1 for users + 100 for posts).

Use the `include` parameter with `select()` to eager-load relationships:

```
$user = new User();
$users = $user->select("*", "", [], "name ASC", 20, 0, ["posts"]);
```

The seventh argument is an array of relationship names to include. This runs just 2 queries (one for users, one for all related posts) and stitches the results together.

toDict() with Nested Includes

When eager loading is active, `toDict()` includes the related data:

```
$user = new User();
$users = $user->select("*", "", [], "", 0, 0, ["posts"]);

$result = array_map(fn($u) => $u->toDict(), $users);

return $response->json($result);
```

```
[
  {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com",
    "posts": [
      {"id": 1, "title": "First Post", "body": "Hello world!"},
      {"id": 3, "title": "Second Post", "body": "Another one."}
    ]
  },
  {
    "id": 2,
    "name": "Bob",
    "email": "bob@example.com",
    "posts": [
      {"id": 2, "title": "Bob's Post", "body": "Hi there."}
    ]
  }
]
```

Nested Eager Loading

Load multiple levels deep with dot notation:

```
$user = new User();
$users = $user->select("*", "", [], "", 0, 0, ["posts", "posts.comments"]);
```

This loads users, their posts, and each post's comments in 3 queries total.

11. Soft Delete

If your model has a `deletedAt` property, Tina4 supports soft delete -- marking records as deleted without actually removing them from the database:

```
<?php
use Tina4\ORM;

class Post extends ORM
{
```

```
    public int $id;
    public string $title;
    public string $body;
    public ?string $deletedAt = null;

    public string $tableName = "posts";
    public string $primaryKey = "id";
    public bool $softDelete = true;
}
```

With `$softDelete = true`:

• `$post->delete()` sets `deleted_at` to the current timestamp instead of deleting the row

• `select()` automatically excludes rows where `deleted_at` is not null

• `$post->forceDelete()` permanently removes the row

Restoring Soft-Deleted Records

```
$post = new Post();
$post->load(5); // Load even if soft-deleted
$post->deletedAt = null;
$post->save();
```

Including Soft-Deleted Records in Queries

```
$post = new Post();
$allPosts = $post->select("*", "", [], "", 0, 0, [], true); // eighth arg = include deleted
```

12. NumericField for Prices

When working with money, floating-point arithmetic causes rounding errors. Use the `NumericField` type for precise decimal values:

```
<?php
use Tina4\ORM;
use Tina4\NumericField;

class Product extends ORM
{
    public int $id;
    public string $name;
    public NumericField $price;
    public NumericField $discount;

    public string $tableName = "products";
    public string $primaryKey = "id";
}
```

`NumericField` maps to `DECIMAL` or `NUMERIC` in the database and handles precision correctly. When you access `$product->price`, you get a value that compares and calculates accurately for financial operations.

13. Auto-CRUD

One of Tina4's most powerful features is auto-CRUD -- it automatically generates REST endpoints for any ORM model.

Add the `$autoCrud` property to your model:

```
<?php
use Tina4\ORM;

class Product extends ORM
{
    public int $id;
    public string $name;
    public string $category = "Uncategorized";
    public float $price = 0.00;
    public bool $inStock = true;

    public string $tableName = "products";
    public string $primaryKey = "id";
    public bool $autoCrud = true;
}
```

With `$autoCrud = true`, Tina4 automatically registers these routes:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/products` | List all with pagination |

| `GET` | `/api/products/{id}` | Get one by ID |

| `POST` | `/api/products` | Create a new record |

| `PUT` | `/api/products/{id}` | Update a record |

| `DELETE` | `/api/products/{id}` | Delete a record |

The endpoint prefix is derived from the table name: `products` becomes `/api/products`.

```
curl http://localhost:7145/api/products
```

```
{
  "data": [
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock": true},
    {"id": 2, "name": "Yoga Mat", "category": "Fitness", "price": 29.99, "in_stock": true}
  ],
  "total": 2,
  "page": 1,
  "per_page": 20
}
```

Auto-CRUD supports query parameters for filtering, sorting, and pagination out of the box:

```
curl "http://localhost:7145/api/products?category=Electronics&sort=price&order=desc&page=1&per_page=10"
```

You can still define custom routes alongside auto-CRUD. Your custom routes take precedence over the auto-generated ones.

14. Exercise: Build a Blog

Build a blog with three models: User, Post, and Comment. Use relationships, eager loading, and auto-CRUD.

Requirements

1. Create three models in `src/orm/`:

User -- `users` table:

• `id` (int, primary key)

• `name` (string)

• `email` (string)

• `createdAt` (string)

• Has many posts

Post -- `posts` table:

• `id` (int, primary key)

• `userId` (int, foreign key)

• `title` (string)

• `body` (string)

• `published` (bool, default false)

• `createdAt` (string)

• Belongs to user, has many comments

Comment -- `comments` table:

• `id` (int, primary key)

• `postId` (int, foreign key)

• `authorName` (string)

• `body` (string)

• `createdAt` (string)

• Belongs to post

2. Create migrations for all three tables.

3. Build custom endpoints:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/blog/posts` | List published posts with author info (eager load user) |

| `GET` | `/api/blog/posts/{id:int}` | Get a post with author and comments (eager load both) |

| `POST` | `/api/blog/posts/{id:int}/comments` | Add a comment to a post |

4. Enable auto-CRUD on the User model for admin access at `/api/users`.

Test with:

```
# Create a user (via auto-CRUD)
curl -X POST http://localhost:7145/api/users \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com"}'
```

```
# Create a post
curl -X POST http://localhost:7145/api/blog/posts \
  -H "Content-Type: application/json" \
  -d '{"user_id": 1, "title": "My First Post", "body": "Hello world!", "published": true}'

# List posts
curl http://localhost:7145/api/blog/posts

# Add a comment
curl -X POST http://localhost:7145/api/blog/posts/1/comments \
  -H "Content-Type: application/json" \
  -d '{"author_name": "Bob", "body": "Great post!"}'

# Get post with comments
curl http://localhost:7145/api/blog/posts/1
```

15. Solution

Migrations

Create `src/migrations/20260322150000_create_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS users;
```

Create `src/migrations/20260322150100_create_posts_table.sql`:

```
-- UP
CREATE TABLE posts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    title TEXT NOT NULL,
    body TEXT NOT NULL,
    published INTEGER NOT NULL DEFAULT 0,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id)
);

-- DOWN
DROP TABLE IF EXISTS posts;
```

Create `src/migrations/20260322150200_create_comments_table.sql`:

```
-- UP
CREATE TABLE comments (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    post_id INTEGER NOT NULL,
    author_name TEXT NOT NULL,
    body TEXT NOT NULL,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (post_id) REFERENCES posts(id)
);

-- DOWN
```

```
DROP TABLE IF EXISTS comments;
```

Run them:

```
tina4 migrate
```

```
Running migrations...
  [APPLIED] 20260322150000_create_users_table.sql
  [APPLIED] 20260322150100_create_posts_table.sql
  [APPLIED] 20260322150200_create_comments_table.sql
Migrations complete. 3 applied.
```

Models

Create `src/orm/User.php`:

```
<?php
use Tina4\ORM;

class User extends ORM
{
    public int $id;
    public string $name;
    public string $email;
    public string $createdAt;

    public string $tableName = "users";
    public string $primaryKey = "id";
    public bool $autoCrud = true;

    public function posts(): array
    {
        return $this->hasMany(Post::class, "user_id");
    }
}
```

Create `src/orm/Post.php`:

```
<?php
use Tina4\ORM;

class Post extends ORM
{
    public int $id;
    public int $userId;
    public string $title;
    public string $body;
    public bool $published = false;
    public string $createdAt;

    public string $tableName = "posts";
    public string $primaryKey = "id";

    public function user(): ?User
    {
        return $this->belongsTo(User::class, "user_id");
    }

    public function comments(): array
    {
        return $this->hasMany(Comment::class, "post_id");
    }
}
```

Create `src/orm/Comment.php`:

```php
<?php
use Tina4\ORM;

class Comment extends ORM
{
    public int $id;
    public int $postId;
    public string $authorName;
    public string $body;
    public string $createdAt;

    public string $tableName = "comments";
    public string $primaryKey = "id";

    public function post(): ?Post
    {
        return $this->belongsTo(Post::class, "post_id");
    }
}
```

Routes

Create `src/routes/blog.php`:

```php
<?php
use Tina4\Route;

// List published posts with author
Route::get("/api/blog/posts", function ($request, $response) {
    $post = new Post();
    $posts = $post->select("*", "published = :published", ["published" => 1], "created_at DESC", 0, 0, ["user

    $results = array_map(fn($p) => $p->toDict(), $posts);

    return $response->json([
        "posts" => $results,
        "count" => count($results)
    ]);
});

// Get a single post with author and comments
Route::get("/api/blog/posts/{id:int}", function ($request, $response) {
    $post = new Post();
    $post->load($request->params["id"]);

    if (empty($post->id)) {
        return $response->json(["error" => "Post not found"], 404);
    }

// ...
```

Expected output for GET /api/blog/posts/1:

```json
{
  "id": 1,
  "user_id": 1,
  "title": "My First Post",
  "body": "Hello world!",
  "published": true,
  "created_at": "2026-03-22 15:00:00",
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
```

```
  },
  "comments": [
    {
      "id": 1,
      "post_id": 1,
      "author_name": "Bob",
      "body": "Great post!",
      "created_at": "2026-03-22 15:01:00"
    }
  ],
  "comment_count": 1
}
```

16. Gotchas

1. Table Naming Convention

Problem: Your model class is `OrderItem` but queries fail because the table does not exist.

Cause: Tina4 converts `OrderItem` to `order_items` (plural, snake_case). If your table is named `order_item` (singular), it will not match.

Fix: Set `$tableName` explicitly: `public string $tableName = "order_item";`. Or rename your table to match the convention.

2. Null Handling

Problem: A field that should be nullable causes errors when the value is null.

Cause: The PHP property is declared as `string` instead of `?string`. PHP 8.1+ enforces type declarations strictly.

Fix: Use nullable types: `public ?string $description = null;`. The `?` prefix allows null values.

3. Relationship Foreign Key Direction

Problem: You write `$this->hasMany(Post::class, "id")` and get wrong results.

Cause: The foreign key argument is the column on the related table, not the current table. `hasMany(Post::class, "user_id")` means "find posts where posts.user_id = this.id", not "find posts where posts.id = this.user_id".

Fix: The foreign key is always on the "many" side. For `hasMany`, it is the column on the child table. For `belongsTo`, it is the column on the current table.

4. camelCase to snake_case Mapping

Problem: You have a property `$userId` but the database column is `user_id`. Queries return null for this field.

Cause: Tina4 automatically converts between `camelCase` (PHP) and `snake_case` (database). If your database column is `userid` (no underscore), the mapping breaks.

Fix: Use consistent naming. PHP properties should be `camelCase` and database columns should be `snake_case`. If your column names differ, you may need to adjust them or override the mapping.

5. Forgetting save()

Problem: You set properties on a model but the database does not change.

Cause: You forgot to call `$model->save()`. Setting properties only changes the in-memory object.

Fix: Always call `save()` after modifying properties that should be persisted.

6. Auto-CRUD Endpoint Conflicts

Problem: Your custom route at `/api/products/{id}` does not work after enabling auto-CRUD on the Product model.

Cause: Both your custom route and the auto-CRUD route match the same path. The first one registered wins.

Fix: Custom routes defined in `src/routes/` files are loaded before auto-CRUD routes, so they take precedence. If that is not the behavior you want, use a different path for your custom route (e.g., `/api/shop/products/{id}`).

7. select() Returns Objects, Not Arrays

Problem: You try to use array syntax (`$result["name"]`) on the result of `select()` and get an error.

Cause: `select()` returns an array of model objects, not associative arrays. Each item is an instance of your model class.

Fix: Access properties with object syntax: `$result->name`. Or convert to an array with `$result->toDict()`.

Chapter 7: Authentication

1. Locking the Door

Up to now, every endpoint you have built is public. Anyone with the URL can read, create, update, and delete data. That is fine for a tutorial, but a real application needs to know who is making a request and whether they are allowed to make it.

This chapter covers Tina4's authentication system: JWT tokens, password hashing, middleware-based route protection, CSRF tokens for forms, and session management.

2. JWT Tokens

Tina4 uses JSON Web Tokens (JWT) for authentication. A JWT is a signed string that contains a payload (like a user ID and role). The server creates the token at login, the client sends it with every request, and the server verifies it without needing to look anything up in a database.

Generating a Token

```
<?php
use Tina4\Auth;

$payload = [
    "user_id" => 42,
    "email" => "alice@example.com",
    "role" => "admin"
];

$token = Auth::getToken($payload);
```

`getToken()` signs the payload with a secret key and returns a JWT string like:


```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo0MiwiZW1haWwiOiJhbGljZUBleGFtcGxlLmNvbSIsInJvbGUiOiJhZG1pb
```


The token has three parts separated by dots: header, payload, and signature. The signature ensures the token has not been tampered with.

Token Expiry

By default, tokens expire after 1 hour. Configure this in `.env`:

```
TINA4_JWT_EXPIRY=3600
```

The value is in seconds. Common settings:

| Value | Duration |

|-------|----------|

| `900` | 15 minutes |

| `3600` | 1 hour (default) |

| `86400` | 24 hours |

| `604800` | 7 days |

Validating a Token

```
$isValid = Auth::validToken($token);
// Returns true if the token is valid and not expired, false otherwise
```

Reading the Payload

```
$payload = Auth::getPayload($token);
```

Returns the original payload array:

```
[
    "user_id" => 42,
    "email" => "alice@example.com",
    "role" => "admin",
    "iat" => 1711112600,  // issued at (Unix timestamp)
    "exp" => 1711116200   // expires at (Unix timestamp)
]
```

If the token is invalid or expired, `getPayload()` returns `null`.

The Secret Key

Tina4 uses a secret key to sign and verify tokens. On first run, it automatically generates a random key and stores it in `secrets/jwt.key`. You can also set it explicitly:

```
TINA4_JWT_SECRET=my-very-long-and-random-secret-key-at-least-32-chars
```

Keep this key secret. If someone gets it, they can forge tokens.

3. Password Hashing

Never store passwords in plain text. Tina4 provides two functions for secure password handling:

Hashing a Password

```
use Tina4\Auth;

$hash = Auth::hashPassword("my-secure-password");
// Returns: "$2y$10$abc123...long-hash-string..."
```

This uses PHP's `password_hash()` with bcrypt under the hood. Each hash includes a random salt, so hashing the same password twice produces different results.

Checking a Password

```
$isCorrect = Auth::checkPassword("my-secure-password", $storedHash);
// Returns true if the password matches the hash
```

Registration Example

```
<?php
use Tina4\Route;
use Tina4\Auth;
use Tina4\Database;

Route::post("/api/register", function ($request, $response) {
    $body = $request->body;

    // Validate input
    if (empty($body["name"]) || empty($body["email"]) || empty($body["password"])) {
        return $response->json(["error" => "Name, email, and password are required"], 400);
    }
```

```
    if (strlen($body["password"]) < 8) {
        return $response->json(["error" => "Password must be at least 8 characters"], 400);
    }

    $db = Database::getConnection();

    // Check if email already exists
    $existing = $db->fetchOne("SELECT id FROM users WHERE email = :email", ["email" => $body["email"]]);
    if ($existing !== null) {
        return $response->json(["error" => "Email already registered"], 409);
    }

// ...
```

```
curl -X POST http://localhost:7145/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful",
  "user": {"id": 1, "name": "Alice", "email": "alice@example.com"}
}
```

4. The Login Flow

Here is the complete login flow: the client sends credentials, the server validates them, and returns a JWT token.

```
<?php
use Tina4\Route;
use Tina4\Auth;
use Tina4\Database;

/**
 * @noauth
 */
Route::post("/api/login", function ($request, $response) {
    $body = $request->body;

    if (empty($body["email"]) || empty($body["password"])) {
        return $response->json(["error" => "Email and password are required"], 400);
    }

    $db = Database::getConnection();

    // Find the user
    $user = $db->fetchOne(
        "SELECT id, name, email, password_hash FROM users WHERE email = :email",
        ["email" => $body["email"]]
    );

    if ($user === null) {
        return $response->json(["error" => "Invalid email or password"], 401);
// ...
```

Notice the `@noauth` annotation. The login endpoint must be public -- you cannot require a token to get a token.

```
curl -X POST http://localhost:7145/api/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
  }
}
```

The client stores this token (in localStorage, a cookie, or memory) and sends it with subsequent requests.

5. Using Tokens in Requests

The client sends the token in the `Authorization` header with the `Bearer` prefix:

```
curl http://localhost:7145/api/profile \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

6. Protecting Routes

Auth Middleware

Create a reusable auth middleware:

```
<?php
use Tina4\Auth;

function authMiddleware($request, $response, $next) {
    $authHeader = $request->headers["Authorization"] ?? "";

    if (empty($authHeader) || !str_starts_with($authHeader, "Bearer ")) {
        return $response->json(["error" => "Authorization header required"], 401);
    }

    $token = substr($authHeader, 7); // Remove "Bearer " prefix

    if (!Auth::validToken($token)) {
        return $response->json(["error" => "Invalid or expired token"], 401);
    }

    $payload = Auth::getPayload($token);
    $request->user = $payload; // Attach user data to the request

    return $next($request, $response);
}
```

Applying Middleware to Routes

```
<?php
use Tina4\Route;

Route::get("/api/profile", function ($request, $response) {
    return $response->json([
        "user_id" => $request->user["user_id"],
        "email" => $request->user["email"],
        "name" => $request->user["name"]
    ]);
}, "authMiddleware");
```

```
# Without token -- 401
curl http://localhost:7145/api/profile

{"error":"Authorization header required"}

# With valid token -- 200
curl http://localhost:7145/api/profile \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

{
  "user_id": 1,
  "email": "alice@example.com",
  "name": "Alice"
}
```

Applying Middleware to a Group

```
Route::group("/api/admin", function () {

    Route::get("/stats", function ($request, $response) {
        return $response->json(["active_users" => 42]);
    });

    Route::get("/logs", function ($request, $response) {
        return $response->json(["logs" => []]);
    });

}, "authMiddleware");
```

Every route in the group requires a valid token.

7. @noauth and @secured Decorators

As introduced in Chapter 2, these decorators control authentication at the route level.

@noauth -- Skip Authentication

Use `@noauth` for public endpoints that should bypass any global or group-level auth:

```
/**
 * @noauth
 */
Route::get("/api/public/health", function ($request, $response) {
    return $response->json(["status" => "ok"]);
});

/**
 * @noauth
 */
Route::post("/api/login", function ($request, $response) {
    // Login logic
});

/**
 * @noauth
 */
Route::post("/api/register", function ($request, $response) {
    // Registration logic
});
```

@secured -- Require Authentication for GET Routes

By default, POST, PUT, PATCH, and DELETE routes are considered secured. GET routes are public. Use `@secured` to explicitly protect a GET route:

```
/**
 * @secured
 */
Route::get("/api/me", function ($request, $response) {
    // This GET route requires authentication
    return $response->json($request->user);
});
```

Role-Based Authorization

Combine auth middleware with role checks:

```
<?php
use Tina4\Auth;

function requireRole($role) {
    return function ($request, $response, $next) use ($role) {
        // First check authentication
        $authHeader = $request->headers["Authorization"] ?? "";
        if (empty($authHeader) || !str_starts_with($authHeader, "Bearer ")) {
            return $response->json(["error" => "Authorization required"], 401);
        }

        $token = substr($authHeader, 7);
        if (!Auth::validToken($token)) {
            return $response->json(["error" => "Invalid or expired token"], 401);
        }

        $payload = Auth::getPayload($token);

        // Check role
        if (($payload["role"] ?? "") !== $role) {
            return $response->json(["error" => "Forbidden. Required role: " . $role], 403);
        }

        $request->user = $payload;
        return $next($request, $response);
// ...
```

Note that `requireRole()` returns a closure. To use it as middleware, you need to register the returned function:

```
$adminOnly = requireRole("admin");

Route::delete("/api/users/{id:int}", function ($request, $response) {
    // Only admins can delete users
    return $response->json(["deleted" => true]);
}, $adminOnly);
```

8. CSRF Protection

For traditional form-based applications (not SPAs), Tina4 provides CSRF protection with form tokens.

Generating a Token

In your template, include the CSRF token in every form:

```
<form method="POST" action="/profile/update">
```

```
    {{ form_token() }}

    <div class="form-group">
        <label for="name">Name</label>
        <input type="text" name="name" id="name" value="{{ user.name }}">
    </div>

    <button type="submit">Update Profile</button>
</form>
```

`{{ form_token() }}` renders a hidden input field:

```
<input type="hidden" name="_token" value="abc123randomtoken456">
```

Validating the Token

In your route handler, check the token:

```
<?php
use Tina4\Route;
use Tina4\Auth;

Route::post("/profile/update", function ($request, $response) {
    // Validate CSRF token
    if (!Auth::validateFormToken($request->body["_token"] ?? "")) {
        return $response->json(["error" => "Invalid form token. Please refresh and try again."], 403);
    }

    // Process the form...
    return $response->redirect("/profile");
});
```

The CSRF token is tied to the user's session and expires after a single use. This prevents cross-site request forgery attacks where a malicious site tricks the user's browser into submitting a form.

When to Use CSRF Tokens

Use CSRF tokens for:

• HTML forms submitted by browsers

• Any POST/PUT/DELETE from server-rendered pages

You do not need CSRF tokens for:

• API endpoints that use JWT (the Bearer token already proves the request is intentional)

• Single-page applications that use `fetch()` with custom headers (the `Authorization` header cannot be set by cross-origin forms)

9. Sessions

Tina4 supports server-side sessions for storing per-user state between requests. Sessions work alongside JWT tokens -- use JWTs for API authentication and sessions for stateful web pages.

Session Configuration

Set the session backend in `.env`:

```
# File-based sessions (default)
TINA4_SESSION_DRIVER=file
```

```
# Redis
TINA4_SESSION_DRIVER=redis
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=6379

# MongoDB
TINA4_SESSION_DRIVER=mongodb
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=27017

# Valkey
TINA4_SESSION_DRIVER=valkey
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=6379
```

File-based sessions work out of the box with no additional dependencies. Use Redis or Valkey for production deployments with multiple servers, so sessions are shared across instances.

Using Sessions

Access session data via `$request->session`:

```php
<?php
use Tina4\Route;

Route::post("/login-form", function ($request, $response) {
    // After validating credentials...
    $request->session["user_id"] = 42;
    $request->session["user_name"] = "Alice";
    $request->session["logged_in"] = true;

    return $response->redirect("/dashboard");
});

Route::get("/dashboard", function ($request, $response) {
    if (empty($request->session["logged_in"])) {
        return $response->redirect("/login");
    }

    return $response->render("dashboard.html", [
        "user_name" => $request->session["user_name"]
    ]);
});

Route::post("/logout", function ($request, $response) {
    // Clear all session data
    $request->session = [];
// ...
```

Session Options

```
TINA4_SESSION_LIFETIME=3600       # Session lifetime in seconds (default: 3600)
TINA4_SESSION_NAME=tina4_session  # Cookie name for the session ID
```

10. Exercise: Build Login, Register, and Profile

Build a complete authentication system with registration, login, profile viewing, and password changing.

Requirements

1. Create a `users` table migration with: `id`, `name`, `email` (unique), `password_hash`, `role` (default "user"), `created_at`

2. Build these endpoints:

| Method | Path | Auth | Description |

|--------|------|------|-------------|

| `POST` | `/api/register` | @noauth | Create an account. Validate name, email, password (min 8 chars). |

| `POST` | `/api/login` | @noauth | Login. Return JWT token. |

| `GET` | `/api/profile` | secured | Get current user's profile from token. |

| `PUT` | `/api/profile` | secured | Update name and email. |

| `PUT` | `/api/profile/password` | secured | Change password. Require current password. |

3. Create auth middleware that extracts the user from the JWT and attaches it to `$request->user`.

Test with:

```
# Register
curl -X POST http://localhost:7145/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'

# Login
curl -X POST http://localhost:7145/api/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securePass123"}'

# Save the token from login response, then:

# Get profile
curl http://localhost:7145/api/profile \
  -H "Authorization: Bearer YOUR_TOKEN_HERE"

# Update profile
curl -X PUT http://localhost:7145/api/profile \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Smith"}'

# Change password
curl -X PUT http://localhost:7145/api/profile/password \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
// ...
```

11. Solution

Migration

Create `src/migrations/20260322160000_create_auth_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
```

```
    role TEXT NOT NULL DEFAULT 'user',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS users;
```

```
tina4 migrate
```

Middleware

Create `src/routes/middleware.php`:

```
<?php
use Tina4\Auth;

function authMiddleware($request, $response, $next) {
    $authHeader = $request->headers["Authorization"] ?? "";

    if (empty($authHeader) || !str_starts_with($authHeader, "Bearer ")) {
        return $response->json(["error" => "Authorization required. Send: Authorization: Bearer <token>"], 401
    }

    $token = substr($authHeader, 7);

    if (!Auth::validToken($token)) {
        return $response->json(["error" => "Invalid or expired token. Please login again."], 401);
    }

    $payload = Auth::getPayload($token);
    $request->user = $payload;

    return $next($request, $response);
}
```

Routes

Create `src/routes/auth.php`:

```
<?php
use Tina4\Route;
use Tina4\Auth;
use Tina4\Database;

/**
 * @noauth
 */
Route::post("/api/register", function ($request, $response) {
    $body = $request->body;

    // Validate input
    $errors = [];
    if (empty($body["name"])) {
        $errors[] = "Name is required";
    }
    if (empty($body["email"])) {
        $errors[] = "Email is required";
    }
    if (empty($body["password"])) {
        $errors[] = "Password is required";
    } elseif (strlen($body["password"]) < 8) {
        $errors[] = "Password must be at least 8 characters";
    }

// ...
```

Expected output for register:

```
{
  "message": "Registration successful",
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com",
    "role": "user",
    "created_at": "2026-03-22 16:00:00"
  }
}
```

(Status: `201 Created`)

Expected output for login:

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com",
    "role": "user"
  }
}
```

Expected output for profile without token:

```
{"error":"Authorization required. Send: Authorization: Bearer <token>"}
```

(Status: `401 Unauthorized`)

Expected output for profile with token:

```
{
  "id": 1,
  "name": "Alice",
  "email": "alice@example.com",
  "role": "user",
  "created_at": "2026-03-22 16:00:00"
}
```

Expected output for password change with wrong current password:

```
{"error":"Current password is incorrect"}
```

(Status: `401 Unauthorized`)

Expected output for successful password change:

```
{"message":"Password changed successfully"}
```

12. Gotchas

1. Token Expiry Confusion

Problem: Tokens that worked yesterday now return 401.

Cause: The default token lifetime is 1 hour. After that, the token is invalid even if the signature is correct.

Fix: Issue a new token at login. If your application needs long-lived sessions, use refresh tokens: a short-lived access token (15 minutes) paired with a long-lived refresh token (7 days) that can be used to get a new access token without re-entering credentials.

2. Secret Key Management

Problem: Tokens generated on one server are invalid on another, or tokens stop working after a deployment.

Cause: Each server generated its own random `secrets/jwt.key` file. Or the key file was deleted/regenerated during deployment.

Fix: Set `TINA4_JWT_SECRET` in `.env` explicitly and use the same value across all servers. Store it in your deployment secrets manager (not in version control). If the key changes, all existing tokens become invalid and users must log in again.

3. CORS with Authentication

Problem: Frontend requests with the `Authorization` header fail with a CORS error, even though `CORS_ORIGINS=*` is set.

Cause: When the browser sends an `Authorization` header, it first sends a preflight `OPTIONS` request. The server must respond to the OPTIONS request with the correct CORS headers, including `Access-Control-Allow-Headers: Authorization`.

Fix: Tina4 handles preflight requests automatically. Make sure `CORS_ORIGINS` is set correctly. If it is still failing, check that you are not overriding CORS headers in middleware.

4. Storing Tokens in localStorage

Problem: Your token is stolen via an XSS attack because it was stored in `localStorage`.

Cause: Any JavaScript on the page can read `localStorage`, including injected scripts from an XSS vulnerability.

Fix: Store tokens in `httpOnly` cookies when possible -- they cannot be accessed by JavaScript. For SPAs that must use `localStorage`, implement strict Content Security Policy headers and sanitize all user input.

5. Forgetting @noauth on Login

Problem: Your login endpoint returns 401 -- you cannot log in because the endpoint requires authentication.

Cause: If you have global auth middleware, the login endpoint needs the `@noauth` annotation to bypass it.

Fix: Add `@noauth` to your login and register routes. Without it, users cannot authenticate because authentication is required to authenticate -- a catch-22.

6. Password Hash Column Too Short

Problem: Registration fails with a database error about the password hash being too long.

Cause: bcrypt hashes are 60 characters long. If your `password_hash` column is defined as `VARCHAR(50)`, it gets truncated.

Fix: Use `TEXT` for the password hash column, or at minimum `VARCHAR(255)`. Never constrain the hash length.

7. Token in URL Query Parameters

Problem: Tokens in URLs like `/api/profile?token=eyJ...` leak through browser history, server logs, and the Referer header.

Cause: Query parameters are visible in many places where headers are not.

Fix: Always send tokens in the `Authorization` header, never in the URL. The only exception is WebSocket connections, where the initial HTTP upgrade request cannot carry custom headers -- use a short-lived token for that case.

Chapter 8: Middleware

1. The Gatekeepers

Your API needs CORS headers for the React frontend, rate limiting for the public endpoints, and auth checking for admin routes -- all without cluttering your route handlers. You could copy-paste the same ten lines of CORS code into every handler, but that breaks the moment you forget one. You could pile all the checks into a giant `if` tree at the top of each handler, but that buries the actual business logic under boilerplate.

Middleware solves this. It lets you wrap routes with reusable logic that runs before (or after) the handler. Each middleware does one job -- check a token, set CORS headers, log the request, enforce rate limits -- and passes control to the next layer. Your route handlers stay focused on their actual purpose.

In Chapter 2 you saw a brief introduction to middleware. This chapter goes deep: built-in middleware, custom middleware, execution order, short-circuiting, and real-world patterns.

2. What Middleware Is

A middleware function sits between the incoming HTTP request and your route handler. It receives the request, the response, and a `$next` function. It can:

1. Inspect or modify the request before passing it along

2. Decide not to pass it along (short-circuit)

3. Inspect or modify the response on the way back

Here is the simplest middleware that does nothing but pass through:

```
<?php

function passthrough($request, $response, $next) {
    return $next($request, $response);
}
```

And here is one that blocks everything:

```
<?php

function blockEverything($request, $response, $next) {
    return $response->json(["error" => "Service unavailable"], 503);
}
```

The `$next` function is what makes middleware composable. Call it to continue to the next middleware (or the route handler if there is no more middleware). Do not call it to stop the chain.

3. Built-in CorsMiddleware

Cross-Origin Resource Sharing (CORS) is the browser mechanism that controls which domains can call your API. When your React frontend at `http://localhost:3000` makes a fetch request to your Tina4 API at `http://localhost:7145`, the browser sends a preflight `OPTIONS` request first. If your API does not respond with the right headers, the browser blocks the request entirely.

Tina4 provides a built-in `CorsMiddleware` that handles this. Configure it in `.env`:

```
CORS_ORIGINS=http://localhost:3000,https://myapp.com
CORS_METHODS=GET,POST,PUT,PATCH,DELETE,OPTIONS
CORS_HEADERS=Content-Type,Authorization,X-API-Key
CORS_MAX_AGE=86400
CORS_CREDENTIALS=true
```

Apply it to a group:

```
<?php
use Tina4\Route;

Route::group("/api", function () {

    Route::get("/products", function ($request, $response) {
        return $response->json(["products" => []]);
    });

    Route::post("/products", function ($request, $response) {
        return $response->json(["created" => true], 201);
    });

}, "CorsMiddleware");
```

Test the preflight:

```
curl -X OPTIONS http://localhost:7145/api/products \
  -H "Origin: http://localhost:3000" \
  -H "Access-Control-Request-Method: POST" \
  -H "Access-Control-Request-Headers: Content-Type,Authorization" \
  -v
```

You should see these response headers:

```
Access-Control-Allow-Origin: http://localhost:3000
Access-Control-Allow-Methods: GET,POST,PUT,PATCH,DELETE,OPTIONS
Access-Control-Allow-Headers: Content-Type,Authorization,X-API-Key
Access-Control-Max-Age: 86400
Access-Control-Allow-Credentials: true
```

The `OPTIONS` request returns `204 No Content` with those headers. The browser caches the preflight result for 86400 seconds (24 hours), so subsequent requests skip the preflight entirely.

Wildcard Origins

During development, you can allow all origins:

```
CORS_ORIGINS=*
```

This is the default when you scaffold a project. Do not use `*` in production -- specify your actual domains.

CORS Without Middleware

If you prefer, you can skip the middleware and handle CORS globally. Add `CORS_ORIGINS` to your `.env` and Tina4 applies CORS headers to all responses automatically. The middleware approach gives you finer control -- you can apply CORS to specific route groups and leave internal routes without CORS headers.

4. Built-in RateLimiter

Rate limiting prevents a single client from overwhelming your API. Tina4's built-in `RateLimiter` middleware tracks requests per IP address and returns `429 Too Many Requests` when the limit is exceeded.

Configure it in `.env`:

```
TINA4_RATE_LIMIT=60
TINA4_RATE_LIMIT_WINDOW=60
```

This means 60 requests per 60 seconds per IP. Apply it:

```
<?php
use Tina4\Route;

Route::group("/api/public", function () {

    Route::get("/search", function ($request, $response) {
        $q = $request->query["q"] ?? "";
        return $response->json(["query" => $q, "results" => []]);
    });

    Route::get("/catalog", function ($request, $response) {
        return $response->json(["items" => []]);
    });

}, "RateLimiter");
```

When a client exceeds the limit:

```
curl http://localhost:7145/api/public/search?q=test
```

```
{"error":"Rate limit exceeded. Try again in 42 seconds.","retry_after":42}
```

The response includes a `Retry-After` header with the number of seconds until the limit resets.

Custom Limits Per Group

You can override the global rate limit for specific groups by setting the limit in the middleware call:

```
<?php
use Tina4\Route;

// Public endpoints: 30 requests per minute
Route::group("/api/public", function () {
    Route::get("/search", function ($request, $response) {
        return $response->json(["results" => []]);
    });
}, "RateLimiter:30");

// Authenticated endpoints: 120 requests per minute
Route::group("/api/v1", function () {
    Route::get("/data", function ($request, $response) {
        return $response->json(["data" => []]);
    });
}, ["authMiddleware", "RateLimiter:120"]);
```

The syntax `"RateLimiter:30"` passes `30` as a parameter to the rate limiter. This overrides the global `TINA4_RATE_LIMIT` for that group only.

5. Writing Custom Middleware

Custom middleware follows the same pattern: receive `$request`, `$response`, and `$next`. Define it as a named function.

Request Logging Middleware

```
<?php

function logRequest($request, $response, $next) {
    $start = microtime(true);
    $method = $request->method;
    $path = $request->path;
    $ip = $request->ip;

    // Log the incoming request
    error_log("[" . date("Y-m-d H:i:s") . "] " . $method . " " . $path . " from " . $ip);

    // Call the next middleware or route handler
    $result = $next($request, $response);

    // Log the completion
    $duration = round((microtime(true) - $start) * 1000, 2);
    error_log("  Completed in " . $duration . "ms");

    return $result;
}
```

Save this in `src/routes/middleware.php` (or any file in `src/routes/`). Apply it:

```
Route::get("/api/products", function ($request, $response) {
    return $response->json(["products" => []]);
}, "logRequest");
```

```
curl http://localhost:7145/api/products
```

Your server log shows:

```
[2026-03-22 14:30:00] GET /api/products from 127.0.0.1
  Completed in 2.34ms
```

Request Timing Middleware

```
<?php

function addTiming($request, $response, $next) {
    $start = microtime(true);

    $result = $next($request, $response);

    $duration = round((microtime(true) - $start) * 1000, 2);

    // Add timing header to the response
    $response->addHeader("X-Response-Time", $duration . "ms");

    return $result;
}
```

```
curl http://localhost:7145/api/products -v
```

In the response headers:

```
X-Response-Time: 3.12ms
```

This is useful for performance monitoring. Your frontend or monitoring tools can read the `X-Response-Time` header to track endpoint performance.

IP Whitelist Middleware

```php
<?php

function ipWhitelist($request, $response, $next) {
    $allowedIps = explode(",", $_ENV["ALLOWED_IPS"] ?? "127.0.0.1");

    if (!in_array($request->ip, $allowedIps)) {
        return $response->json([
            "error" => "Access denied",
            "your_ip" => $request->ip
        ], 403);
    }

    return $next($request, $response);
}
```

Configure in `.env`:

```
ALLOWED_IPS=127.0.0.1,10.0.0.5,192.168.1.100
```

Apply it to admin routes:

```php
Route::group("/admin", function () {
    Route::get("/dashboard", function ($request, $response) {
        return $response->json(["page" => "admin dashboard"]);
    });
}, "ipWhitelist");
```

```
curl http://localhost:7145/admin/dashboard
```

If your IP is in the whitelist:

```
{"page":"admin dashboard"}
```

If not:

```
{"error":"Access denied","your_ip":"203.0.113.42"}
```

Request Validation Middleware

```php
<?php

function requireJson($request, $response, $next) {
    if (in_array($request->method, ["POST", "PUT", "PATCH"])) {
        $contentType = $request->headers["Content-Type"] ?? "";

        if (strpos($contentType, "application/json") === false) {
            return $response->json([
                "error" => "Content-Type must be application/json",
                "received" => $contentType
            ], 415);
        }
    }

    return $next($request, $response);
}
```

This middleware ensures all POST, PUT, and PATCH requests send JSON. Apply it to your entire API:

```php
Route::group("/api", function () {
    // All API routes here
}, "requireJson");
```

```
curl -X POST http://localhost:7145/api/products \
  -d "name=Widget"
```

```
{"error":"Content-Type must be application/json","received":"application/x-www-form-urlencoded"}
```

(Status: `415 Unsupported Media Type`)

6. Applying Middleware to Individual Routes

Pass middleware as the third argument to any route method:

```
<?php
use Tina4\Route;

Route::get("/api/data", function ($request, $response) {
    return $response->json(["data" => [1, 2, 3]]);
}, "logRequest");

Route::post("/api/data", function ($request, $response) {
    return $response->json(["created" => true], 201);
}, ["logRequest", "requireJson"]);
```

For a single middleware, pass a string. For multiple, pass an array. Each middleware runs in the order listed.

7. Route Groups with Shared Middleware

Groups apply middleware to every route inside them:

```
<?php
use Tina4\Route;

// Public API -- rate limited, CORS enabled
Route::group("/api/public", function () {

    Route::get("/products", function ($request, $response) {
        return $response->json(["products" => []]);
    });

    Route::get("/categories", function ($request, $response) {
        return $response->json(["categories" => []]);
    });

}, ["CorsMiddleware", "RateLimiter:30"]);

// Admin API -- auth required, IP restricted, logged
Route::group("/api/admin", function () {

    Route::get("/users", function ($request, $response) {
        return $response->json(["users" => []]);
    });

    Route::delete("/users/{id:int}", function ($request, $response) {
        $id = $request->params["id"];
// ...
```

Individual routes inside a group can add their own middleware too. The group middleware runs first, then the route-specific middleware:

```
Route::group("/api", function () {

    // This route gets: logRequest -> requireJson -> handler
```

```
    Route::post("/upload", function ($request, $response) {
        return $response->json(["uploaded" => true]);
    }, "requireJson");

}, "logRequest");
```

8. Middleware Execution Order

When you stack middleware, they execute from outer to inner -- like layers of an onion. The first middleware listed runs first on the way in and last on the way out.

Consider this setup:

```
<?php

function middlewareA($request, $response, $next) {
    error_log("A: before");
    $result = $next($request, $response);
    error_log("A: after");
    return $result;
}

function middlewareB($request, $response, $next) {
    error_log("B: before");
    $result = $next($request, $response);
    error_log("B: after");
    return $result;
}

function middlewareC($request, $response, $next) {
    error_log("C: before");
    $result = $next($request, $response);
    error_log("C: after");
    return $result;
}

Route::get("/api/test", function ($request, $response) {
    error_log("Handler");
    return $response->json(["ok" => true]);
}, ["middlewareA", "middlewareB", "middlewareC"]);

curl http://localhost:7145/api/test
```

Server log output:

```
A: before
B: before
C: before
Handler
C: after
B: after
A: after
```

The request flows inward: A, B, C, Handler. The response flows outward: C, B, A. This means:

• `middlewareA` sees the request first and the response last. Put CORS headers and request logging here.

• `middlewareC` sees the request last and the response first. Put response transformations here.

• Authentication should be early (catch unauthorized requests before doing expensive work).

• Timing should be outermost (measure the entire pipeline).

Group + Route Middleware Order

```
Route::group("/api", function () {

    Route::get("/data", function ($request, $response) {
        return $response->json(["data" => true]);
    }, "middlewareC");

}, ["middlewareA", "middlewareB"]);
```

Execution order: `middlewareA` -> `middlewareB` -> `middlewareC` -> handler. Group middleware always runs before route middleware.

9. Short-Circuiting

When middleware does not call `$next`, the chain stops. No subsequent middleware runs and the route handler is never called. This is called short-circuiting.

Authentication Short-Circuit

```
<?php

function requireAuth($request, $response, $next) {
    $token = $request->headers["Authorization"] ?? "";

    if (empty($token)) {
        // Short-circuit: return 401, do not call $next
        return $response->json(["error" => "Authentication required"], 401);
    }

    return $next($request, $response);
}
```

If the `Authorization` header is missing, the middleware returns a `401` immediately. The route handler never executes. This saves server resources -- you do not query the database or run business logic for unauthenticated requests.

Maintenance Mode

```
<?php

function maintenanceMode($request, $response, $next) {
    $isMaintenanceMode = ($_ENV["MAINTENANCE_MODE"] ?? "false") === "true";

    if ($isMaintenanceMode) {
        // Allow health checks even in maintenance mode
        if ($request->path === "/health") {
            return $next($request, $response);
        }

        return $response->json([
            "error" => "Service is undergoing maintenance",
            "retry_after" => 300
        ], 503);
    }

    return $next($request, $response);
}
```

Add to `.env` when you need to take the site down:

```
MAINTENANCE_MODE=true
```

Apply it globally:

```
Route::group("/", function () {
    // All your routes
}, "maintenanceMode");
```

Every request except `/health` gets a `503 Service Unavailable` response.

Conditional Short-Circuiting

Sometimes you want to short-circuit only certain methods:

```
<?php

function readOnly($request, $response, $next) {
    if (in_array($request->method, ["POST", "PUT", "PATCH", "DELETE"])) {
        return $response->json([
            "error" => "API is in read-only mode"
        ], 405);
    }

    return $next($request, $response);
}
```

GET requests pass through. Write operations are blocked. This is useful for standby replicas or demo environments.

10. Modifying Requests in Middleware

Middleware can add data to the request before passing it to the handler. This is how authentication middleware attaches user information:

```
<?php
use Tina4\Auth;

function attachUser($request, $response, $next) {
    $authHeader = $request->headers["Authorization"] ?? "";

    if (!empty($authHeader) && str_starts_with($authHeader, "Bearer ")) {
        $token = substr($authHeader, 7);
        if (Auth::validToken($token)) {
            $request->user = Auth::getPayload($token);
        }
    }

    // Always call $next -- this middleware does not block requests
    // Routes that need auth should check $request->user themselves
    return $next($request, $response);
}
```

This pattern is different from a blocking auth middleware. It attaches user data if a token is present but does not block unauthenticated requests. Some routes need the user data (show personalized content) but are still accessible without it.

Adding Request Metadata

```
<?php

function addRequestId($request, $response, $next) {
```

```
    $requestId = bin2hex(random_bytes(8));
    $request->requestId = $requestId;

    $result = $next($request, $response);

    // Add request ID to response headers for debugging
    $response->addHeader("X-Request-Id", $requestId);

    return $result;
}
```

Your route handler can now access `$request->requestId` for logging, error tracking, or correlation:

```
Route::get("/api/data", function ($request, $response) {
    error_log("[" . $request->requestId . "] Processing data request");
    return $response->json(["request_id" => $request->requestId, "data" => []]);
}, "addRequestId");
```

```
curl http://localhost:7145/api/data -v
```

```
{"request_id":"a1b2c3d4e5f6a7b8","data":[]}
```

Response headers include:

```
X-Request-Id: a1b2c3d4e5f6a7b8
```

11. Real-World Middleware Stack

Here is a realistic middleware setup for a production API:

```
<?php
use Tina4\Route;

// src/routes/middleware.php

function addRequestId($request, $response, $next) {
    $request->requestId = bin2hex(random_bytes(8));
    $result = $next($request, $response);
    $response->addHeader("X-Request-Id", $request->requestId);
    return $result;
}

function logRequest($request, $response, $next) {
    $start = microtime(true);
    error_log("[" . $request->requestId . "] " . $request->method . " " . $request->path);

    $result = $next($request, $response);

    $duration = round((microtime(true) - $start) * 1000, 2);
    error_log("[" . $request->requestId . "] Completed in " . $duration . "ms");

    return $result;
}

function requireApiKey($request, $response, $next) {
// ...
```

```
<?php
use Tina4\Route;

// src/routes/api.php

Route::group("/api/v1", function () {
```

```
    Route::get("/products", function ($request, $response) {
        return $response->json(["products" => [
            ["id" => 1, "name" => "Widget", "price" => 9.99],
            ["id" => 2, "name" => "Gadget", "price" => 19.99]
        ]]);
    });

    Route::get("/products/{id:int}", function ($request, $response) {
        $id = $request->params["id"];
        return $response->json(["id" => $id, "name" => "Widget", "price" => 9.99]);
    });

    Route::post("/products", function ($request, $response) {
        $body = $request->body;
        return $response->json([
            "id" => 3,
            "name" => $body["name"] ?? "Unknown",
            "price" => (float) ($body["price"] ?? 0)
// ...
```

Test without an API key:

```
curl http://localhost:7145/api/v1/products
```

```
{"error":"Invalid or missing API key","request_id":"a1b2c3d4e5f6a7b8"}
```

(Status: `401 Unauthorized`)

Test with a valid API key:

```
curl http://localhost:7145/api/v1/products \
  -H "X-API-Key: my-secret-key"
```

```
{"products":[{"id":1,"name":"Widget","price":9.99},{"id":2,"name":"Gadget","price":19.99}]}
```

12. Exercise: Build an API Key Middleware

Build a middleware called `validateApiKey` that:

1. Checks for an `X-API-Key` header on every request

2. Validates the key against a comma-separated list stored in the `API_KEYS` environment variable

3. If the key is missing, returns `401` with `{"error": "API key required"}`

4. If the key is invalid, returns `403` with `{"error": "Invalid API key"}`

5. If the key is valid, attaches the key to `$request->apiKey` and continues

6. Apply this middleware to a route group with at least two endpoints

Setup

Add this to your `.env`:

```
API_KEYS=key-alpha-001,key-beta-002,key-gamma-003
```

Test with:

```
# No API key -- should get 401
curl http://localhost:7145/api/partner/data

# Invalid API key -- should get 403
```

```
curl http://localhost:7145/api/partner/data \
  -H "X-API-Key: wrong-key"

# Valid API key -- should get 200
curl http://localhost:7145/api/partner/data \
  -H "X-API-Key: key-alpha-001"

# Valid key on another endpoint
curl http://localhost:7145/api/partner/stats \
  -H "X-API-Key: key-beta-002"
```

13. Solution

Create `src/routes/api-key-middleware.php`:

```
<?php
use Tina4\Route;

function validateApiKey($request, $response, $next) {
    $apiKey = $request->headers["X-API-Key"] ?? "";

    // Check if header is present
    if (empty($apiKey)) {
        return $response->json(["error" => "API key required"], 401);
    }

    // Validate against allowed keys
    $validKeys = array_map("trim", explode(",", $_ENV["API_KEYS"] ?? ""));

    if (!in_array($apiKey, $validKeys)) {
        return $response->json(["error" => "Invalid API key"], 403);
    }

    // Attach key to request for downstream use
    $request->apiKey = $apiKey;

    return $next($request, $response);
}

Route::group("/api/partner", function () {
// ...
```

Expected output -- no key:

```
curl http://localhost:7145/api/partner/data
```

```
{"error":"API key required"}
```

(Status: `401 Unauthorized`)

Expected output -- invalid key:

```
curl http://localhost:7145/api/partner/data -H "X-API-Key: wrong-key"
```

```
{"error":"Invalid API key"}
```

(Status: `403 Forbidden`)

Expected output -- valid key:

```
curl http://localhost:7145/api/partner/data -H "X-API-Key: key-alpha-001"
```

```
{
```

```
  "authenticated_with": "key-alpha-001",
  "data": [
    {"id": 1, "value": "alpha"},
    {"id": 2, "value": "beta"}
  ]
}
```

Expected output -- different valid key on stats:

```
curl http://localhost:7145/api/partner/stats -H "X-API-Key: key-beta-002"
```

```
{
  "authenticated_with": "key-beta-002",
  "stats": {
    "total_requests": 1423,
    "avg_response_ms": 42
  }
}
```

14. Gotchas

1. Middleware Must Be a Named Function

Problem: You pass an anonymous closure as middleware and get an error.

Cause: Tina4 expects middleware to be referenced by a string name, not as an inline closure. The string is resolved to a function at runtime.

Fix: Define your middleware as a named function: `function myMiddleware($request, $response, $next) { ... }` and pass `"myMiddleware"` as a string.

2. Forgetting to Return $next()

Problem: Your middleware runs but the route handler never executes. The response is empty or a 500 error.

Cause: You called `$next($request, $response)` but did not `return` the result.

Fix: Always `return $next($request, $response)`. Without the `return`, the middleware discards the response from the handler and returns `null`.

3. Middleware Order Matters

Problem: Your logging middleware does not see the request ID, even though `addRequestId` is in the middleware list.

Cause: `logRequest` runs before `addRequestId`. Middleware executes in the order listed.

Fix: Put `addRequestId` before `logRequest` in the array: `["addRequestId", "logRequest"]`. Think of the order as "what needs to happen first."

4. CORS Preflight Returns 404

Problem: The browser's preflight `OPTIONS` request gets a 404, but `GET` and `POST` work fine when tested with curl.

Cause: You did not apply `CorsMiddleware` to the route, so the `OPTIONS` method is not handled.

Fix: Apply `CorsMiddleware` to the group. It automatically handles `OPTIONS` requests and returns the correct CORS headers. Alternatively, set `CORS_ORIGINS` in `.env` for global CORS handling.

5. Rate Limiter Counts Preflight Requests

Problem: Your frontend hits the rate limit faster than expected because every `POST` request actually counts as two requests (the `OPTIONS` preflight plus the `POST`).

Cause: The rate limiter counts all requests, including `OPTIONS`.

Fix: Put `CorsMiddleware` before `RateLimiter` in the middleware chain: `["CorsMiddleware", "RateLimiter"]`. The CORS middleware handles `OPTIONS` and returns immediately, so the rate limiter only sees actual requests. If CORS is configured globally via `.env`, the preflight is handled before middleware runs.

6. Middleware File Not Auto-Loaded

Problem: You defined middleware in a file but get "function not found" when referencing it.

Cause: The file is not in `src/routes/`. Tina4 auto-loads all `.php` files in `src/routes/`, but middleware defined outside that directory is not discovered.

Fix: Put your middleware functions in a file inside `src/routes/`, such as `src/routes/middleware.php`. The filename does not matter -- all `.php` files in the directory are loaded.

7. Short-Circuiting Skips Cleanup Middleware

Problem: Your timing middleware logs the start time but never logs the completion time for blocked requests.

Cause: When an inner middleware short-circuits, the outer middleware's code after `$next()` still runs. But if the short-circuiting middleware is listed before the timing middleware, the timing middleware never executes at all.

Fix: Put cleanup-dependent middleware (timing, logging) at the outermost layer. They wrap the entire chain, so they always see both the start and the end -- even when an inner middleware short-circuits. The code after `$next()` in outer middleware always runs, regardless of whether the chain was short-circuited.

Chapter 15: Frontend with tina4css

1. The Problem with Frontend Toolchains

Your client wants a beautiful dashboard. You know the drill -- install Node.js, run `npm install`, wait for 200MB of `node_modules` to download, configure webpack or Vite, set up PostCSS, add a CSS framework, maybe Tailwind with its purge config, and then pray nothing breaks when you upgrade a dependency six months from now.

Tina4 takes a different approach. The framework ships with **tina4css** -- a Bootstrap-compatible CSS framework -- and **frond.js** -- a lightweight JavaScript helper library. Both are included automatically when you scaffold a project. No npm. No webpack. No build step for development. Just link the files and start building.

By the end of this chapter, you will have a complete admin dashboard with a sidebar, navigation, cards, tables, modals, and dark mode support -- all without touching npm.

2. What Ships with Tina4

When you run `tina4 init`, two files appear in your project:

```
src/public/
■■■ css/
■   ■■■ tina4.css        # The CSS framework
■■■ js/
■   ■■■ frond.js         # AJAX helpers, form submission, token management
■■■ scss/
    ■■■ tina4.scss       # SCSS source (optional, for customization)
```

Include them in any template:

```
<link rel="stylesheet" href="/css/tina4.css">
<script src="/js/frond.js"></script>
```

That is all you need. No CDN, no package manager, no version conflicts.

3. The Grid System

tina4css uses a 12-column responsive grid, compatible with Bootstrap's class names. If you already know Bootstrap, you already know tina4css.

```
<div class="container">
    <div class="row">
        <div class="col-md-4">
            <p>One third</p>
        </div>
        <div class="col-md-4">
            <p>One third</p>
        </div>
        <div class="col-md-4">
            <p>One third</p>
        </div>
    </div>
</div>
```

Breakpoints:

| Class Prefix | Screen Width | Typical Device |

|----------|----------|----------|

| `col-` | All sizes | Phones and up |

| `col-sm-` | >= 576px | Large phones |

| `col-md-` | >= 768px | Tablets |

| `col-lg-` | >= 992px | Desktops |

| `col-xl-` | >= 1200px | Large desktops |

Columns stack vertically on screens smaller than their breakpoint. A `col-md-6` element takes half the row on tablets and up, but full width on phones.

4. Components

Navbar

```
<nav class="navbar navbar-dark bg-dark">
    <div class="container">
        <a class="navbar-brand" href="/">My Dashboard</a>
        <ul class="navbar-nav">
            <li class="nav-item">
                <a class="nav-link active" href="/dashboard">Dashboard</a>
            </li>
            <li class="nav-item">
                <a class="nav-link" href="/users">Users</a>
            </li>
            <li class="nav-item">
                <a class="nav-link" href="/settings">Settings</a>
            </li>
        </ul>
    </div>
</nav>
```

Use `navbar-light bg-light` for a light theme, or `navbar-dark bg-primary` for a colored background.

Buttons

```
<button class="btn btn-primary">Save</button>
<button class="btn btn-secondary">Cancel</button>
<button class="btn btn-danger">Delete</button>
<button class="btn btn-success">Approve</button>
<button class="btn btn-warning">Warning</button>
<button class="btn btn-outline-primary">Outlined</button>
<button class="btn btn-sm btn-primary">Small</button>
<button class="btn btn-lg btn-primary">Large</button>
```

Cards

```
<div class="card">
    <div class="card-header">
        Monthly Revenue
    </div>
    <div class="card-body">
        <h5 class="card-title">$12,450</h5>
        <p class="card-text">Up 8% from last month</p>
```

```
    </div>
    <div class="card-footer text-muted">
        Updated 5 minutes ago
    </div>
</div>
```

Forms

```
<form>
    <div class="form-group">
        <label for="email">Email address</label>
        <input type="email" class="form-control" id="email"
               placeholder="you@example.com">
    </div>
    <div class="form-group">
        <label for="password">Password</label>
        <input type="password" class="form-control" id="password">
    </div>
    <div class="form-group form-check">
        <input type="checkbox" class="form-check-input" id="remember">
        <label class="form-check-label" for="remember">Remember me</label>
    </div>
    <button type="submit" class="btn btn-primary">Sign In</button>
</form>
```

Tables

```
<table class="table table-striped table-hover">
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>Role</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>1</td>
            <td>Alice Johnson</td>
            <td>alice@example.com</td>
            <td><span class="badge badge-primary">Admin</span></td>
            <td>
                <button class="btn btn-sm btn-outline-primary">Edit</button>
                <button class="btn btn-sm btn-outline-danger">Delete</button>
            </td>
        </tr>
        <tr>
            <td>2</td>
            <td>Bob Smith</td>
            <td>bob@example.com</td>
// ...
```

Table variants: `table-bordered`, `table-striped`, `table-hover`, `table-sm` (compact), `table-responsive` (wraps in a scrollable container on small screens).

Badges

```
<span class="badge badge-primary">Primary</span>
<span class="badge badge-success">Active</span>
<span class="badge badge-warning">Pending</span>
<span class="badge badge-danger">Overdue</span>
<span class="badge badge-info">Info</span>
```

```
<span class="badge badge-dark">Dark</span>
```

Alerts

```
<div class="alert alert-success">
    Product saved successfully.
</div>

<div class="alert alert-danger">
    Error: could not connect to the database.
</div>

<div class="alert alert-warning alert-dismissible">
    Your trial expires in 3 days.
    <button type="button" class="close" data-dismiss="alert">&times;</button>
</div>

<div class="alert alert-info">
    Tip: you can drag and drop items to reorder them.
</div>
```

Modals

```
<button class="btn btn-primary" data-toggle="modal" data-target="#confirmModal">
    Delete User
</button>

<div class="modal" id="confirmModal">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title">Confirm Deletion</h5>
                <button type="button" class="close" data-dismiss="modal">&times;</button>
            </div>
            <div class="modal-body">
                <p>Are you sure you want to delete this user? This action cannot be undone.</p>
            </div>
            <div class="modal-footer">
                <button class="btn btn-secondary" data-dismiss="modal">Cancel</button>
                <button class="btn btn-danger">Delete</button>
            </div>
        </div>
    </div>
</div>
```

tina4css includes the JavaScript needed for modal toggling. No jQuery required.

Progress Bars

```
<div class="progress">
    <div class="progress-bar bg-success" style="width: 75%">75%</div>
</div>

<div class="progress">
    <div class="progress-bar bg-warning progress-bar-striped" style="width: 45%">
        45% - Uploading...
    </div>
</div>
```

5. SCSS Customization

The SCSS source lives in `src/public/scss/tina4.scss`. If you want to customize colors, fonts, or spacing, edit this file and compile it to CSS.

Variables

At the top of `tina4.scss`, you will find variables you can override:

```
// Colors
$primary:   #007bff;
$secondary: #6c757d;
$success:   #28a745;
$danger:    #dc3545;
$warning:   #ffc107;
$info:      #17a2b8;
$dark:      #343a40;
$light:     #f8f9fa;

// Typography
$font-family-base: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
$font-size-base: 1rem;
$line-height-base: 1.5;

// Spacing
$spacer: 1rem;

// Border radius
$border-radius: 0.25rem;

// Breakpoints
$breakpoint-sm: 576px;
$breakpoint-md: 768px;
$breakpoint-lg: 992px;
// ...
```

Compiling SCSS

If you edit the SCSS files, compile them with the Tina4 CLI:

```
tina4 scss
```

```
Compiling SCSS...
  src/public/scss/tina4.scss → src/public/css/tina4.css
Done.
```

This reads all `.scss` files from `src/public/scss/` and writes compiled CSS to `src/public/css/`. No Sass gem, no Node, no additional dependencies -- the Tina4 CLI includes a built-in SCSS compiler.

You can also run it in watch mode during development:

```
tina4 scss --watch
```

```
Watching src/public/scss/ for changes...
  [14:30:05] Compiled tina4.scss → tina4.css (42ms)
```

Every time you save a `.scss` file, it recompiles automatically.

Custom Theme Example

Create `src/public/scss/custom.scss`:

```
// Override variables before importing tina4
$primary: #6f42c1;
$font-family-base: "Inter", sans-serif;
$border-radius: 0.5rem;
```

```
// Import the base framework
@import "tina4";

// Add your own styles
.sidebar {
    background: $dark;
    color: $light;
    min-height: 100vh;
    padding: 1rem;

    .nav-link {
        color: rgba(255, 255, 255, 0.7);
        padding: 0.5rem 1rem;
        border-radius: $border-radius;

        &:hover, &.active {
            color: #fff;
            background: rgba(255, 255, 255, 0.1);
        }
    }
// ...
```

Compile:

```
tina4 scss
```

Then reference your custom CSS in your template:

```
<link rel="stylesheet" href="/css/custom.css">
```

6. frond.js -- The JavaScript Helper

frond.js is a lightweight JavaScript library that ships with Tina4. It provides AJAX helpers, form submission, and JWT token management without requiring jQuery, Axios, or any other library.

AJAX Requests

```
// GET request
frond.get("/api/products", function (data) {
    console.log(data.products);
});

// GET with error handling
frond.get("/api/products", function (data) {
    console.log(data);
}, function (error) {
    console.error("Failed:", error);
});

// POST request
frond.post("/api/products", {
    name: "Widget",
    price: 9.99
}, function (data) {
    console.log("Created:", data);
});

// PUT request
frond.put("/api/products/1", {
    name: "Updated Widget",
    price: 12.99
```

```
}, function (data) {
// ...
```

Form Submission

frond.js can intercept form submissions and send them via AJAX instead of a full page reload:

```
<form id="createProduct" data-frond-submit="/api/products" data-frond-method="POST">
    <div class="form-group">
        <label for="name">Product Name</label>
        <input type="text" class="form-control" name="name" id="name">
    </div>
    <div class="form-group">
        <label for="price">Price</label>
        <input type="number" class="form-control" name="price" id="price" step="0.01">
    </div>
    <button type="submit" class="btn btn-primary">Create Product</button>
</form>

<div id="result"></div>

<script>
    frond.onSubmit("createProduct", function (response) {
        document.getElementById("result").innerHTML =
            '<div class="alert alert-success">Product created: ' + response.name + '</div>';
    }, function (error) {
        document.getElementById("result").innerHTML =
            '<div class="alert alert-danger">Error: ' + error.message + '</div>';
    });
</script>
```

The `data-frond-submit` attribute tells frond.js the URL to POST to. The `data-frond-method` attribute specifies the HTTP method. frond.js serializes all form fields as JSON automatically.

Token Management

When your application uses JWT authentication (Chapter 7), frond.js manages tokens automatically:

```
// Store the token after login
frond.setToken("eyJhbGciOiJIUzI1NiIs...");

// All subsequent requests include the Authorization header automatically
frond.get("/api/profile", function (data) {
    // The request included: Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
    console.log(data);
});

// Clear the token on logout
frond.clearToken();
```

frond.js stores the token in `localStorage` and includes it as a `Bearer` token in the `Authorization` header on every request.

Loading Indicators

```
// Show a loading state while fetching
frond.get("/api/products", function (data) {
    renderProducts(data.products);
}, null, {
    loading: "#loadingSpinner"  // CSS selector for loading element
});
```

The element with id `loadingSpinner` will be shown while the request is in flight and hidden when it completes.

7. Building a Dashboard Layout

Let us build a real admin dashboard. We will create a template layout with a sidebar, a top navbar, and a content area.

The Base Layout

Create `src/templates/admin/layout.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Admin Dashboard{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; }

        .app-wrapper { display: flex; min-height: 100vh; }

        .sidebar {
            width: 250px;
            background: #1a1a2e;
            color: #e0e0e0;
            padding: 0;
            flex-shrink: 0;
        }
        .sidebar-header {
            padding: 20px;
            font-size: 1.2em;
            font-weight: bold;
            color: #fff;
// ...
```

The Dashboard Page

Create `src/templates/admin/dashboard.html`:

```
{% extends "admin/layout.html" %}

{% block title %}Dashboard - Admin{% endblock %}
{% block page_title %}Dashboard{% endblock %}

{% block content %}
    <div class="stats-grid">
        {% for stat in stats %}
            <div class="stat-card">
                <div class="stat-value">{{ stat.value }}</div>
                <div class="stat-label">{{ stat.label }}</div>
                <div class="stat-change {{ stat.direction }}">
                    {{ stat.direction == "up" ? "+" : "" }}{{ stat.change }}% from last month
                </div>
            </div>
        {% endfor %}
    </div>
```

```
    <div class="row">
        <div class="col-md-8">
            <div class="card">
                <div class="card-header">Recent Orders</div>
                <div class="card-body" style="padding: 0;">
                    <table class="table table-hover" style="margin: 0;">
                        <thead>
// ...
```

The Dashboard Route

Create `src/routes/admin.php`:

```
<?php
use Tina4\Route;

Route::get("/admin", function ($request, $response) {
    $stats = [
        ["label" => "Total Users", "value" => "1,247", "change" => 12, "direction" => "up"],
        ["label" => "Revenue", "value" => "$34,500", "change" => 8, "direction" => "up"],
        ["label" => "Orders", "value" => "456", "change" => 3, "direction" => "down"],
        ["label" => "Conversion Rate", "value" => "3.2%", "change" => 0.5, "direction" => "up"]
    ];

    $recentOrders = [
        ["id" => 1042, "customer" => "Alice Johnson", "total" => 149.99, "status" => "completed"],
        ["id" => 1041, "customer" => "Bob Smith", "total" => 89.50, "status" => "pending"],
        ["id" => 1040, "customer" => "Carol Davis", "total" => 234.00, "status" => "completed"],
        ["id" => 1039, "customer" => "David Lee", "total" => 45.99, "status" => "cancelled"],
        ["id" => 1038, "customer" => "Eva Martinez", "total" => 178.25, "status" => "pending"]
    ];

    return $response->render("admin/dashboard.html", [
        "active_page" => "dashboard",
        "user_name" => "Admin",
        "pending_orders" => 2,
        "stats" => $stats,
        "recent_orders" => $recentOrders,
// ...
```

Start the server and visit `http://localhost:7145/admin`. You should see:

• A dark sidebar on the left with navigation links

• A top bar with a welcome message and logout button

• Four stat cards showing users, revenue, orders, and conversion rate

• A recent orders table with color-coded status badges

• Quick action buttons in a card on the right

• System health progress bars

All of this with zero npm dependencies.

8. Dark Mode Support

tina4css includes built-in dark mode. Add the `data-theme="dark"` attribute to the `<html>` element:

```
<html lang="en" data-theme="dark">
```

This inverts the color scheme -- light backgrounds become dark, dark text becomes light, and all components adapt automatically.

Toggle with JavaScript

Add a dark mode toggle button to your layout:

```
<button class="btn btn-sm btn-outline-secondary" onclick="toggleDarkMode()">
    Toggle Dark Mode
</button>

<script>
    function toggleDarkMode() {
        const html = document.documentElement;
        const current = html.getAttribute("data-theme");
        const next = current === "dark" ? "light" : "dark";
        html.setAttribute("data-theme", next);
        localStorage.setItem("theme", next);
    }

    // Load saved preference
    const saved = localStorage.getItem("theme");
    if (saved) {
        document.documentElement.setAttribute("data-theme", saved);
    }
</script>
```

Respecting System Preference

If the user has not set a preference, respect their operating system setting:

```
const saved = localStorage.getItem("theme");
if (saved) {
    document.documentElement.setAttribute("data-theme", saved);
} else if (window.matchMedia("(prefers-color-scheme: dark)").matches) {
    document.documentElement.setAttribute("data-theme", "dark");
}
```

Custom Dark Mode Colors

In your SCSS, you can customize the dark mode palette:

```
[data-theme="dark"] {
    --bg-primary: #1a1a2e;
    --bg-secondary: #16213e;
    --text-primary: #e0e0e0;
    --text-secondary: #a0a0a0;
    --border-color: #2d2d4a;
    --card-bg: #16213e;
}
```

9. Responsive Design

tina4css is mobile-first. Components stack vertically on small screens by default and expand horizontally on larger screens.

Responsive Utilities

Show and hide elements at different breakpoints:

```
<!-- Only visible on medium screens and up -->
```

```
<div class="d-none d-md-block">
    Desktop sidebar content
</div>

<!-- Only visible on small screens -->
<div class="d-block d-md-none">
    Mobile navigation
</div>
```

Responsive Tables

Wrap tables in a `.table-responsive` container to make them scroll horizontally on small screens:

```
<div class="table-responsive">
    <table class="table">
        <!-- Wide table content -->
    </table>
</div>
```

Spacing Utilities

Use margin and padding utilities that respond to breakpoints:

```
<div class="p-2 p-md-4">Padding: 0.5rem on mobile, 1.5rem on desktop</div>
<div class="mt-2 mt-lg-5">Margin-top: 0.5rem on mobile, 3rem on large screens</div>
```

Spacing scale:

| Class | Size |

|-------|------|

| `*-0` | 0 |

| `*-1` | 0.25rem |

| `*-2` | 0.5rem |

| `*-3` | 1rem |

| `*-4` | 1.5rem |

| `*-5` | 3rem |

Directions: `m` (margin), `p` (padding), `t` (top), `b` (bottom), `l` (left), `r` (right), `x` (horizontal), `y` (vertical).

10. Building a Users Page with AJAX

Let us build a users management page that loads data via AJAX using frond.js.

Create `src/templates/admin/users.html`:

```
{% extends "admin/layout.html" %}

{% block title %}Users - Admin{% endblock %}
{% block page_title %}User Management{% endblock %}

{% block content %}
    <div class="card">
        <div class="card-header" style="display: flex; justify-content: space-between; align-items: center;">
            <span>All Users</span>
            <button class="btn btn-sm btn-primary" data-toggle="modal" data-target="#addUserModal">
```

```
                Add User
            </button>
        </div>
        <div class="card-body" style="padding: 0;">
            <div id="loadingSpinner" class="text-center p-4" style="display: none;">
                Loading...
            </div>
            <table class="table table-hover" id="usersTable" style="margin: 0;">
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Name</th>
                        <th>Email</th>
                        <th>Created</th>
                        <th>Actions</th>
// ...
```

Create the route for the users page:

```
Route::get("/admin/users", function ($request, $response) {
    return $response->render("admin/users.html", [
        "active_page" => "users",
        "user_name" => "Admin",
        "pending_orders" => 2
    ]);
});
```

This page loads users via an AJAX call to `/api/users` (provided by auto-CRUD on the User model from Chapter 6). The table populates dynamically, and users can be added through a modal form and deleted with confirmation -- all without full page reloads.

11. Exercise: Build a Complete Admin Dashboard

Build an admin dashboard with the following features:

Requirements

1. **Layout** -- Sidebar with navigation links for Dashboard, Products, and Orders. Top bar with the current user name and a dark mode toggle button.

2. **Dashboard Page** (`GET /admin`) -- Four stat cards (Total Products, Total Orders, Revenue, Low Stock Items). A table of the 5 most recent orders. A progress bar showing inventory health (percentage of products in stock).

3. **Products Page** (`GET /admin/products`) -- A table listing all products with columns: ID, Name, Category, Price, Stock Status. An "Add Product" button that opens a modal with a form. The form submits via frond.js AJAX to `POST /api/products`. After submission, the table reloads without a page refresh.

4. **Dark Mode** -- The toggle button switches between light and dark themes. The preference persists in localStorage.

Seed Data

Create a route that populates sample data:

```
Route::get("/admin/seed", function ($request, $response) {
    $products = [
        ["name" => "Wireless Keyboard", "category" => "Electronics", "price" => 79.99, "in_stock" => true],
        ["name" => "USB-C Hub", "category" => "Electronics", "price" => 49.99, "in_stock" => true],
```

```
        ["name" => "Standing Desk", "category" => "Furniture", "price" => 549.99, "in_stock" => false],
        ["name" => "Monitor Light", "category" => "Electronics", "price" => 39.99, "in_stock" => true],
        ["name" => "Ergonomic Chair", "category" => "Furniture", "price" => 399.99, "in_stock" => false]
    ];

    foreach ($products as $data) {
        $product = new Product();
        $product->name = $data["name"];
        $product->category = $data["category"];
        $product->price = $data["price"];
        $product->inStock = $data["in_stock"];
        $product->save();
    }

    return $response->json(["message" => "Seeded " . count($products) . " products"]);
});
```

Expected Result

Visit `http://localhost:7145/admin` and see:

• A sidebar with navigation (Dashboard highlighted)

• Four stat cards showing product and order counts

• A recent orders table with status badges

• A dark mode toggle that switches the entire UI

Visit `http://localhost:7145/admin/products` and see:

• A table of all products loaded via AJAX

• An "Add Product" button that opens a modal

• Form submission creates the product and refreshes the table

12. Solution

The solution follows the same patterns shown in sections 7 and 10. Create the layout template extending the base layout from section 7. Create the dashboard page template with stat cards using `{% for stat in stats %}`. Create the products page template using the AJAX pattern from section 10, but for products instead of users.

For the products page, use auto-CRUD on the Product model (`$autoCrud = true`) so the API endpoints are available at `/api/products`. Load the table with `frond.get("/api/products", ...)` and handle form submission with `frond.post("/api/products", ...)`.

For the dark mode toggle, add the JavaScript from section 8 to the base layout template inside the `{% block extra_js %}` block.

The key route handlers are:

```
<?php
use Tina4\Route;

Route::get("/admin", function ($request, $response) {
    $product = new Product();
    $allProducts = $product->select("*");
    $inStockCount = count(array_filter($allProducts, fn($p) => $p->inStock));
```

```
    $totalProducts = count($allProducts);
    $stockHealth = $totalProducts > 0 ? round(($inStockCount / $totalProducts) * 100) : 0;

    $stats = [
        ["label" => "Total Products", "value" => $totalProducts, "change" => 5, "direction" => "up"],
        ["label" => "Total Orders", "value" => 156, "change" => 12, "direction" => "up"],
        ["label" => "Revenue", "value" => "$8,450", "change" => 3, "direction" => "up"],
        ["label" => "Low Stock", "value" => $totalProducts - $inStockCount, "change" => 2, "direction" => "dow
    ];

    return $response->render("admin/dashboard.html", [
        "active_page" => "dashboard",
        "stats" => $stats,
        "stock_health" => $stockHealth,
        "recent_orders" => [],
        "pending_orders" => 0
    ]);
});
// ...
```

13. Gotchas

1. CSS File Not Loading

Problem: The browser shows unstyled HTML. The CSS file returns a 404.

Cause: The CSS file must be inside `src/public/css/`. If you put it in `src/css/` or `css/` at the root, Tina4 will not serve it.

Fix: Make sure the file is at `src/public/css/tina4.css`. Static files are served from `src/public/`.

2. SCSS Changes Not Reflected

Problem: You edited `tina4.scss` but the browser still shows the old styles.

Cause: SCSS must be compiled to CSS. Editing the `.scss` file does not automatically update the `.css` file unless you are running `tina4 scss --watch`.

Fix: Run `tina4 scss` to compile, or use `tina4 scss --watch` during development.

3. Modal Does Not Open

Problem: Clicking a button with `data-toggle="modal"` does nothing.

Cause: frond.js is not loaded, or the `data-target` value does not match the modal's `id`.

Fix: Check that `<script src="/js/frond.js"></script>` is included before `</body>`. Verify that `data-target="#myModal"` matches `<div class="modal" id="myModal">` (the `#` prefix is required in `data-target`).

4. frond.js AJAX Returns HTML Instead of JSON

Problem: `frond.get("/api/products", ...)` receives an HTML page instead of JSON data.

Cause: The route handler returns `$response->render(...)` or `$response->html(...)` instead of `$response->json(...)`. Or the URL is wrong and hitting a catch-all route that returns HTML.

Fix: Verify the API route returns JSON. Check the URL in the frond.js call matches the route path exactly.

5. Dark Mode Flickers on Load

Problem: The page loads in light mode for a split second before switching to dark mode.

Cause: The theme is set by JavaScript after the page renders. The initial HTML does not include `data-theme="dark"`.

Fix: Move the theme detection script to the `<head>` section so it runs before the body renders:

```
<head>
    <script>
        (function() {
            var t = localStorage.getItem("theme");
            if (t) document.documentElement.setAttribute("data-theme", t);
            else if (window.matchMedia("(prefers-color-scheme: dark)").matches)
                document.documentElement.setAttribute("data-theme", "dark");
        })();
    </script>
</head>
```

6. Form Data Not Reaching the Server

Problem: `frond.post()` sends the request but `$request->body` is empty on the server.

Cause: frond.js sends JSON by default. If you are building the data object incorrectly or passing a FormData object, the server may not parse it correctly.

Fix: Pass a plain JavaScript object to `frond.post()`. Do not use `new FormData()` -- frond.js handles serialization internally. Make sure your object keys match what the server expects:

```
frond.post("/api/products", {
    name: document.getElementById("name").value,
    price: parseFloat(document.getElementById("price").value)
}, callback);
```

7. Responsive Grid Not Working

Problem: Columns do not stack on mobile. They overflow horizontally instead.

Cause: Missing the viewport meta tag. Without it, mobile browsers render the page at desktop width.

Fix: Add this to your `<head>`:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

This tag is included in all the examples above but is easy to forget when creating templates from scratch.